

Tab 1

Markdown

> **System Target:** C++ / SFML or SDL2 / Custom Physics & Rendering Engine
> **LLM Implementation Guidelines:** Read directly as technical instructions for incremental code generation. Perform task implementation sequentially without skipping steps or substituting geometric primitives for finished artwork/shaders.

[Tangential Friction] [Fluid Boundary]
(Slime: $\mu = 0$, Dry: $\mu = 0.8$) (Viscosity, Buoyancy, Drag)

$$F_{net} = F_{gravity} + F_{buoyancy} + F_{drag} + F_{thrust}$$





(Drawn by Blood Field) (Infects Surrounding Host)

Interconnected Ecosystem Matrix (Sample Ecosystem: The Weeds / Atlantis)

1. Merman Interactions:

2.

- Hunts Barracudas and Kelp-Seahorses.
-
- Attacks Tiger Sharks in groups, but flees if isolated.
-
- Spears Electric Rays $\rightarrow$ **Chain Event:** Spear conducts electric discharge $\rightarrow$ Merman is stunned/killed $\rightarrow$ Spawns **Merman Corpse** + **Electric Arc Area**.
-

3. Corpse & Scavenger Cascades:

4.

- **Merman Corpse** releases a expanding **Blood Cloud Field** into the Eulerian scent grid.
-
- Tiger Sharks within 50 meters drop current tasks to enter **Scavenge Mode**, converging on the corpse.
-
- Fungal Parasites infect the corpse; if left unconsumed for 10 seconds, it bursts into a **Spore Cloud** that infects nearby living Barracudas or Mermen, turning them hostiles.
-

5. Electric Ray Death Cascade:

6.

- When killed by any predator, the Ray discharges its internal battery: 3-meter radial electric shock.
-
- Surrounding small prey (Seahorses, Phytoplankton) die instantly, creating a floating debris pile that attracts Scavenger Crabs from the sea floor.
-

3. PROCEDURAL ANIMATION & ACTIVE RAGDOLL IK ENGINE

No frame-by-frame sprites are used. Creatures are animated using physics joints, active motor torques, and Inverse Kinematics (FABRIK).

3.1 FABRIK Implementation with Joint Angle Constraints

C++

```
struct BoneJoint {  
  
    float  
    float // Relative to parent joint  
    float  
  
class ConstrainedFABRIKSolver {  
public  
    static void SolveChain(std::vector<BoneJoint> &joints, float targetPos)  
    {  
        size_t n = joints.size();  
        if (n < 2) return;  
  
        // 1. Check reachability  
        float maxDist = 0.0f;  
        for (size_t i = 0; i < n - 1; i++)  
        {  
            float d = joints[i].GetDistTo(joints[i + 1]);  
            if (d > maxDist) maxDist = d;  
        }  
        if (maxDist > targetPos)
```

```

        for (size_t i = 0; i < 1; ++i)
        {
            // ...

            return;
        }

// 2. Iterative FABRIK Pass
int i = 0;
while (i < 8)
{
    // Backward Pass

    for (int j = static_cast<int>(2); j > 0; --j)
    {
        // ...

        // Forward Pass with Angular Constraints
        for (size_t k = 0; k < 1; ++k)
        {
            // Apply Angular Constraint relative to parent segment angle
            if (i == 0)
            {
                // ...

                // ...
            }
        }
    }
}

private:
static void ConstrainAngle(float &angle, float parentAngle)
{
    float diff = atan2(angle, parentAngle);
    float wrapAngle = atan2(1, 0);

    // Wrap to [-PI, PI]
    while (diff < -3.14159f) diff += 6.28318f;
    while (diff > 3.14159f) diff -= 6.28318f;

    // Clamp angle
    float clampedAngle = std::clamp(angle, parentAngle - wrapAngle, parentAngle + wrapAngle);
}

```

```
float
return cos      sin
```

4. ADVANCED PERCEPTION & UTILITY AI ENGINE

Creatures decide actions using a continuous **Utility Evaluator** weighted by perception fields and personality traits.

4.1 Scent Grid Diffusion Engine (Eulerian Grid)

Scent, blood, and spores diffuse through a downsampled $W \times H$ scalar grid using Discrete Diffusion Equations:

$$C_{x,y}^{t+1} = C_{x,y}^t + \Big(D \cdot \nabla^2 C_{x,y}^t - k_{\text{decay}} \cdot C_{x,y}^t + \vec{v}_{\text{fluid}} \cdot \nabla C_{x,y}^t \Big) \Delta t$$

Entities evaluate scent gradient direction via central differencing:

$$\vec{G}_{scent} = \left(\frac{C_{x+1,y} - C_{x-1,y}}{2 \cdot \Delta x}, \frac{C_{x,y+1} - C_{x,y-1}}{2 \cdot \Delta y} \right)$$

4.2 Utility Action Calculation & The 12 Abnormal Personalities

The action score $U(A)$ for an available behavior $A \in \{Hunt, Flee, Scavenge, Rest, Investigate\}$ is computed as:

$$U(A) = \left(\sum_i w_i \cdot f_i(s) \right) \cdot \text{PersonalityModifier}(A)$$

C++

```
struct PersonalityProfile {
    float // [0.0, 1.0]
    float // [0.0, 1.0]
    float // [0.0, 1.0]
    float // [0.0, 1.0]

class UtilityEngine {
public
    static float CalculateHuntUtility const const const

    float // [0.0 = Satiated, 1.0 = Starving]
    float
    float

    if 1.2f 0.6f return 0.0f // Won't attack larger prey unless brave

    float 1.0f
    float 0.5f 0.3f 0.2f

    // Apply Abnormal Overrides
    switch
    case
        return 1.0f // Always max hunt utility regardless of size/hunger
    case
        return 2.5f
    case
        return 0.0f // Replaces hunting with item stealing
    default
        return std 0.0f 1.0f
```

Detailed Behavioral Execution Specification

C++

```
class BehaviorStateMachine {
public
    static void UpdateEntityBehavior
        // 1. Process Senses
```

return

// 3. Standard Utility Evaluation

float

float

float

```
// 4. Select Dominant Utility Action
```

if $\frac{1}{2} \leq \frac{1}{2} \leq 0.5f$

else if 0.4f

else if 0.3f

else

private

```
static void ExecuteAbnormalBehavior
```

switch

case

```
// Attacks nearest target regardless of species, suffers continuous health loss
```

1.5f

break

case

```
// Tracks player's loose relics or inventory items, grabs, and flees to level boundary
```

if

if $\frac{1}{\lambda} \leq \frac{1}{\lambda_0}$ then $\lambda = \frac{1}{\lambda_0}$ else $\lambda = \frac{1}{\lambda}$ end if

```
// Shift to standard flee
```

break

case

// Peaceful until player/predator approaches home Den (< 6.0m), then acquires
hyper-speed

if

6.0f

2.2f

else

break

case

// Seeks explosive triggers (gunpowder barrels, electrical arcs) near targets

if

if

0.8f

break

case

// Latches onto larger entities or player, injecting control vectors into their inputs

if

if

0.5f

break

case

// Collects loose items and drags them to its nest

if

else

break

case

// Emits light/speed boost aura and follows player closely

```

                                -1.5f -1.0f
                                4.0f
break

case
    // Dominates a 12m radius zone. Attacks ANY entity entering
    for auto
        if
            12.0f

        return

break

case
    // Drawn directly to electrical arcs, light pulses, or powder trails

break

case
    // Consumes anything smaller than itself, gaining 5% mass per consumption
    if

        if
            0.8f

            1.05f
            1.03f

break

case
    // Panics when ambient light < 0.3, fleeing toward nearest bioluminescent source
    if
        0.3f

        1.8f

break

case
    // Emits continuous command wave, forcing all same-species within 15m to join its target
attack
    15.0f

```

break

5. VISUAL ART DIRECTION & ENVIRONMENT

RENDERING SPECIFICATION

CRITICAL ARCHITECTURAL DIRECTIVE FOR CLAUDE CODE:

NEVER generate plain geometric boxes, monochrome placeholder rectangles, or untextured debug polygons for level terrain, entities, or props.

All level geometry must be rendered using **Auto-Tiled Textures**, **Normal Maps**, **Custom Shaders**, **Multi-Layer Parallax Scrolling**, and **Volumetric Particle Systems**.

5.1 Auto-Tiling System (Dual-Grid / Bitmask 8-Bit Rule)

Level terrain surfaces are rendered by evaluating surrounding cell states using an 8-bit bitmask rule to select correct decorative edge tiles (cracked metal, barnacle crusts, overgrown moss, carved Atlantean marble).

```
C++
uint8_t ComputeTileBitmask(const int x, const int y)
{
    uint8_t bitmask = 0;
    if (x > 0 && y > 0) bitmask |= 1; // North
    if (x > 0 && y > 1) bitmask |= 2; // North-East
    if (x > 1 && y > 0) bitmask |= 4; // East
    if (x > 1 && y > 1) bitmask |= 8; // South-East
    if (x > 0 && y > 2) bitmask |= 16; // South
    if (x > 1 && y > 2) bitmask |= 32; // South-West
    if (x > 2 && y > 0) bitmask |= 64; // West
    if (x > 2 && y > 1) bitmask |= 128; // North-West
    return bitmask;
}
```

5.2 Biome Visual Identity & Palette Specifications

Biome	Color Palette (Hex)	Primary Tile Textures	Visual Shader Effects	Environmental Props
1. The Pipes	#1A1C23 (Cast Iron)	Riveted iron plates, rusted pipe elbows, brass steam valves	Heat shimmer distortion, dripping condensation particles	Pressure gauges, escaping steam vents, hanging chains
	#D96B27 (Rust)			
	#4E8752 (Slime)			
2. The Hull	#0F2027 (Deep Ocean)	Barnacle-encrusted metal, rotting copper plating, kelp roots	Water caustic overlay shader, floating plankton particles	Broken ship framing, submerged anchors, hanging ropes

	#2C5364 (Submerged Steel)			
	#E5A93C (Barnacles)			
3. The Pirate Ship	#2C1D11 (Aged Oak) #C0A060 (Rope Canvas) #8B0000 (Crimson Sails)	Dark oak deck planks, weathered wood grain, brass trim	Dynamic lighting, sway distortion on rigging elements	Canvas sails, cannon ports, wooden barrels, ship wheel

4. The Island	#0B2B11 (Jungle Canopy) #5C3D2E (Mud/Bark) #00F0FF (Tropical Water)	Muddy rock faces, tropical vine overlays, mossy wood	Sun shaft volumetric god-rays, falling leaf particles	Tribal totems, rope bridges, coconut clusters, bamboo spikes
5. The Cave	#0D0E15 (Basalt) #2A3B50 (Moist Slate)	Wet basalt, jagged stalactite tiles, glowing spore veins	Echolocation sound-ripple wireframe overlay	Hanging stalactites, glowing mushroom clusters, stone arches

	#00FF88 (Biolum Fungus)			
6. The Weeds	#051A12 (Abyssal Green) #1A5E3A (Kelp Fronds) #FF0055 (Coral Hazards)	Layered kelp stalks, brain coral platforms, sponge walls	Multi-layer fluid sway shader, depth fog vignette	Swaying giant kelp, floating spore pods, sea anemone clusters

7. Atlantis	#112233 (Sunken Basalt) #44AA99 (Patina Copper) #FFD700 (Glowing Runes)	Carved marble blocks, golden runic inlay tiles, aqueduct channels	Runic glow pulsing shader, dynamic water refraction	Sunken statues, clockwork gear mechanisms, broken arches
8. The Open Abyss	#020208 (Void Black) #00E5FF (Cyan Glass)	Translucent Glass Sponges, slimy sheer trench walls	Edge pressure vignette, darkness sound-as-light shader	Black smoker vents, brine pool shelves, Siphonophore webs

	#A000FF (Abyssal Magenta)			
--	---	--	--	--

5.3 GLSL Water Caustics & Fluid Distortion Shader ([water_caustics.frag](#))

OpenGL Shading Language

```
#version 330 core
in vec2
out vec4

uniform sampler2D
uniform float
uniform vec2

void
    vec2

    // Calculate sine-wave offset grid for underwater distortion
    float          sin          20.0      3.0  0.003
    float          cos          15.0      2.5  0.003
        vec2

    // Dynamic Caustic Pattern
    float          sin          40.0      40.0      4.0
        cos          20.0      30.0      2.0
            clamp pow          0.8 3.0 0.0 1.0 0.15

    vec4          texture
    vec3          vec3 0.05 0.25 0.35 // Abyssal teal tint

        vec4 mix          0.25  vec3
```

6. GRANULAR IMPLEMENTATION ROADMAP FOR CLAUDE CODE (OPUS 5.5 IN C++)

Claude Code must implement this architecture in **discrete, non-destructive technical phases**.

Phase 1: Core Physics & Traversal Mechanics

- **Step 1.1:** Write basic math types (`Vector2D`, `Matrix2x2`, `Ray2D`) and velocity-verlet integrator functions.
-
- **Step 1.2:** Implement `PlayerPhysicsController` with full support for state transitions: Ground Sprint, Ground Slide, Impact Roll, Hydro-Glide, Water Dash, and Parachute Brake.
-
- **Step 1.3:** Build narrowphase collision solver handling tile map collision manifolds and variable friction materials.
-

Phase 2: Procedural Tilemap & Generator

- **Step 2.1:** Implement `CellularAutomata` generator for Cave/Pipes biomes.
-
- **Step 2.2:** Build vertical streaming chunk engine for *The Open Abyss* with sliding generation windows.
-
- **Step 2.3:** Write Poisson-Disc Den spawner placement system.
-

Phase 3: IK Rigging & Procedural Locomotion

- **Step 3.1:** Implement `ConstrainedFABRIKSolver` with parent-relative angle limits.
-
- **Step 3.2:** Write locomotion algorithms: Serpentine Sine Wave, Multipied Tripod Gait, Hydrodynamic Swimming Forces, and Biped PD Motors.
-

Phase 4: Scent Diffusion Grid & Utility Perception Engine

- **Step 4.1:** Write downsampled $W \times H$ Eulerian grid class handling scent diffusion, decay, and gradient calculations.
-
- **Step 4.2:** Build **PerceptionSystem** computing raycast vision cones, acoustic wave intensity, and scent gradient detection.
-
- **Step 4.3:** Build **UtilityEngine** computing action preference scores based on traits ($U \in [0.0, 1.0]$).
-

Phase 5: Interconnected Food Web & Behavior Logic

- **Step 5.1:** Implement **FoodWebGraph** managing directed interaction edges, dynamic preference scores, and state transitions (Corpse, Stunned, Infected).
-
- **Step 5.2:** Program the 12 Abnormal Personality overrides within **BehaviorStateMachine**.
-
- **Step 5.3:** Program procedural AI logic for all 80 distinct entities across the 8 biomes.
-

Phase 6: Shaders, Auto-Tiling, Parallax & Polish

- **Step 6.1:** Implement 8-bit dual-grid bitmask auto-tiling system for terrain surfaces.
-
- **Step 6.2:** Integrate GLSL custom shaders (**water_caustics.frag**, **bioluminescence.frag**, **pressure_vignette.frag**).
-
- **Step 6.3:** Build multi-layer parallax rendering engine with depth fog.
-

7. ARCHITECTURAL QUESTIONS & PROPOSALS FOR EXPANSION

Before finalizing code generation for Phase 1, consider these architectural decisions:

1. **Corpse Rot Mechanics & Disease:** Should rotting corpses left unconsumed in water gradually degrade overall biome water visibility (adding a dynamic darkness haze shader) or stay localized to scent clouds?
- 2.
3. **Entity Limb Severance / Active Ragdoll Dismemberment:** When high-mass predators (e.g., Tiger Sharks, Abyssal Leviathans) bite smaller targets, should the FABRIK IK chain sever into separate active rigid-body pieces that retain independent scent emission and attraction?
- 4.
5. **Player Customization & Relic Mechanics:** Should relics collected in high-difficulty areas (like *The Pirate Ship* or *Atlantis*) grant passive physics overrides to the player?

(e.g., permanent reduction in fluid drag, immunity to electric arcs, or scent-masking abilities)?

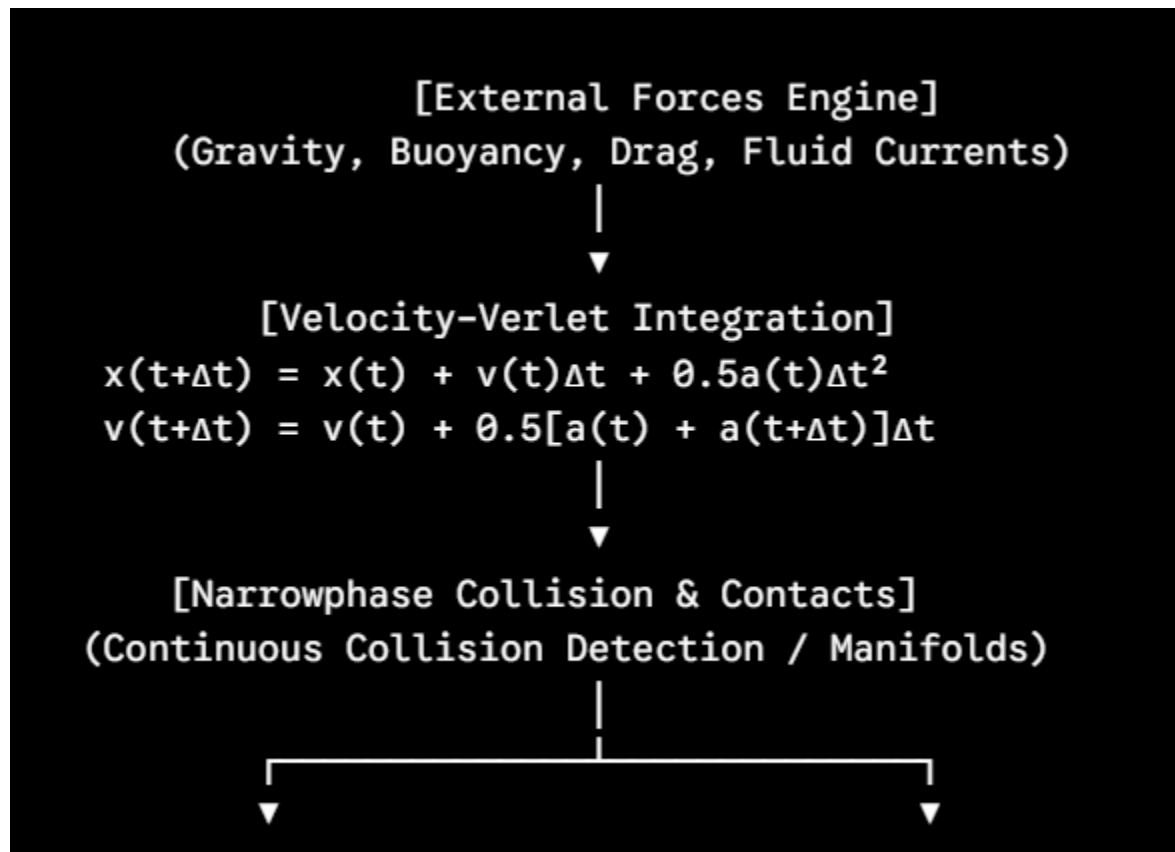
Initial

DIRECTIONS1: STANDALONE PARKOUR ENGINE ARCHITECTURE & SPECIFICATION

> ****System Target:**** C++ / SFML or SDL2 / Custom Physics & Rendering Engine
> ****LLM Implementation Guidelines:**** Read directly as technical instructions for incremental code generation. Perform task implementation sequentially without skipping steps or substituting geometric primitives for finished artwork/shaders.

1. ADVANCED ENGINE PHYSICS & DYNAMIC TRAVERSAL

The physics subsystem uses an integrated ****Impulse-Based Velocity-Verlet Integrator**** coupled with a 2D Eulerian fluid grid to enable smooth transitions between land platforming and 3D hydrodynamic traversal.



[Tangential Friction] [Fluid Boundary] (Slime: $\mu = 0$, Dry: $\mu = 0.8$) (Viscosity, Buoyancy, Drag)

1.1 Mathematical Formulation of Traversal Physics ##### Fluid Drag & Buoyancy Vectors
Every entity (player or fauna) calculates net fluid acceleration using:

$$F_{net} = F_{gravity} + F_{buoyancy} + F_{drag} + F_{thrust}$$

Gravity: $F_{gravity} = m \cdot g \cdot \hat{j}$ **Buoyancy:** $F_{buoyancy} = -\rho_{medium} \cdot V_{submerged} \cdot g \cdot \hat{j}$

Standard Water Density: $\rho_{water} = 1000kg/m^3$ * Heavy Abyssal Brine Density:

$\rho_{brine} = 2400kg/m^3$ (Generates positive upward force for lightweight entities, negative buoyancy

for dense entities). **Hydrodynamic Drag (Quadratic):** $F_{drag} = \frac{1}{2} \cdot \rho_{medium} \cdot C_D \cdot A \cdot |\vec{v}_{rel}| \cdot \vec{v}_{rel}$ Where $\vec{v}_{rel} = \vec{v}_{entity} - \vec{v}_{fluid_current}$, C_D is the dynamic drag coefficient, and A is the frontal

cross-sectional area derived from orientation relative to velocity. ##### Traversal States &

Physics Constants | Traversal State | Drag Coeff (C_D) | Friction (μ) | Gravity Scale | Impulse

Mechanics & Force Inputs | :--- | :--- | :--- | :--- | :--- | **Ground Sprint** | `0.05` | `0.80` | `1.00` |

$F_{drive} = m \cdot a_{max} \cdot \hat{d}$ | **Slimy Wall Slide** | `0.01` | `0.00` | `0.35` | Direct downward sliding;

zero wall jumps possible | **Ground Slide** | `0.02` | `0.05` | `1.00` | Down + Jump applies

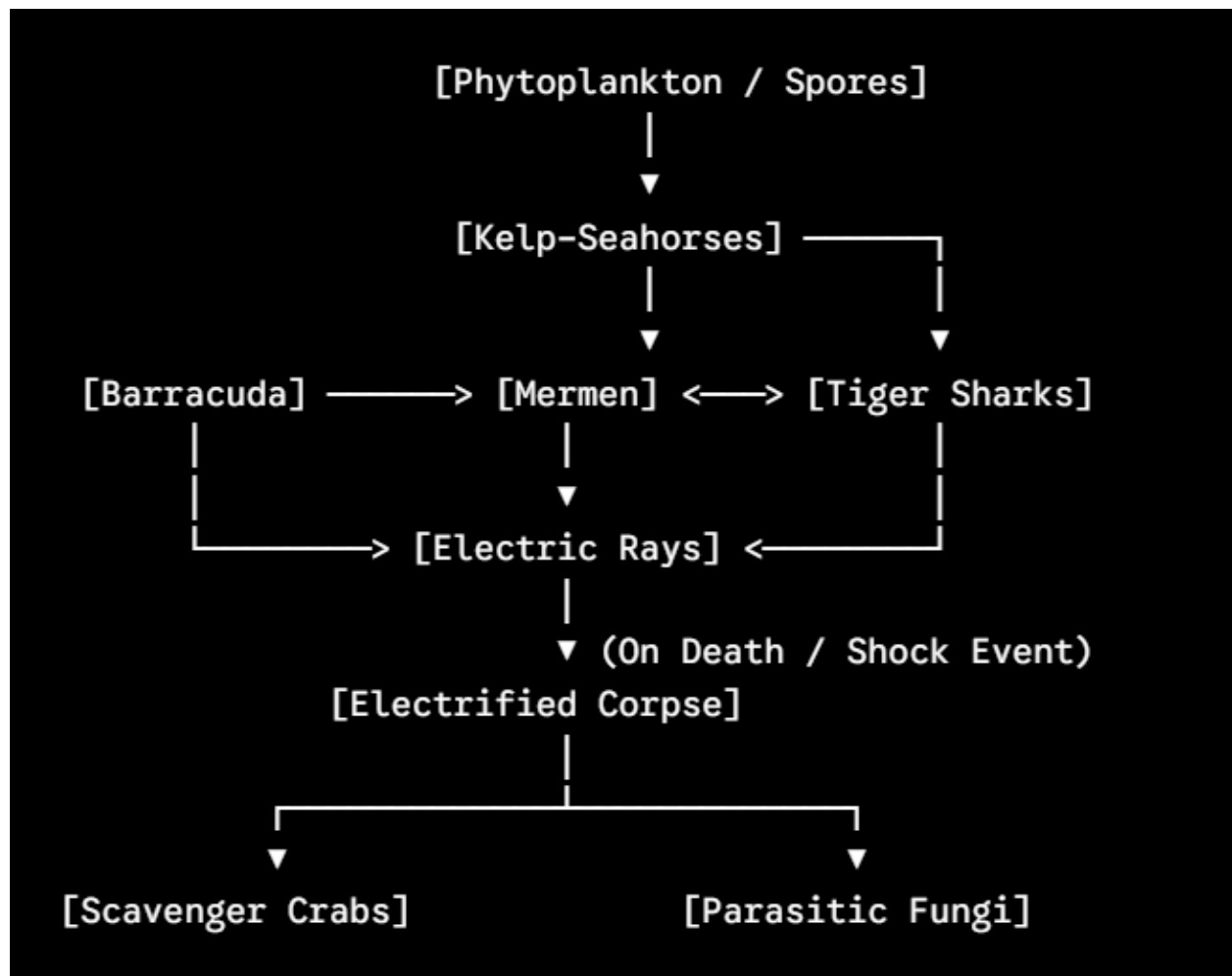
boost: $\vec{I} = 1.8 \cdot m \cdot v_x \cdot \hat{i}$ | **Impact Roll** | `0.03` | `0.10` | `0.50` | Down input converts

vertical fall momentum: $v_x' = v_x + 0.7 \cdot |v_y| \cdot \frac{v_y}{|v_y|}$ | **Fluid Hydro-Glide** | `0.08` | `0.01` | `0.05` | C_D reduced by 65%. Locks into ambient current vectors | **Water Dash** |

`0.80` (post-dash) | `0.00` | `0.00` | Applies directional impulse: $\vec{I}_{dash} = 22.0 \cdot m \cdot \hat{u}$ |

Parachute Brake | `3.50` | `0.00` | `0.20` | Dynamic cross-sectional expansion: flattens A by 400% | --- ## 2. GRAPH-BASED FOOD WEB & COMPLEX ECOSYSTEM ENGINE Entities

do not follow a simple linear predator-prey ladder ($A \rightarrow B \rightarrow C$). Instead, the ecosystem operates as a **Directed Interconnected Graph Network** $G = (V, E)$, where vertices V represent entity states (alive species, corpses, severed limbs, spores) and edges E represent interaction vectors governed by dynamic weights.



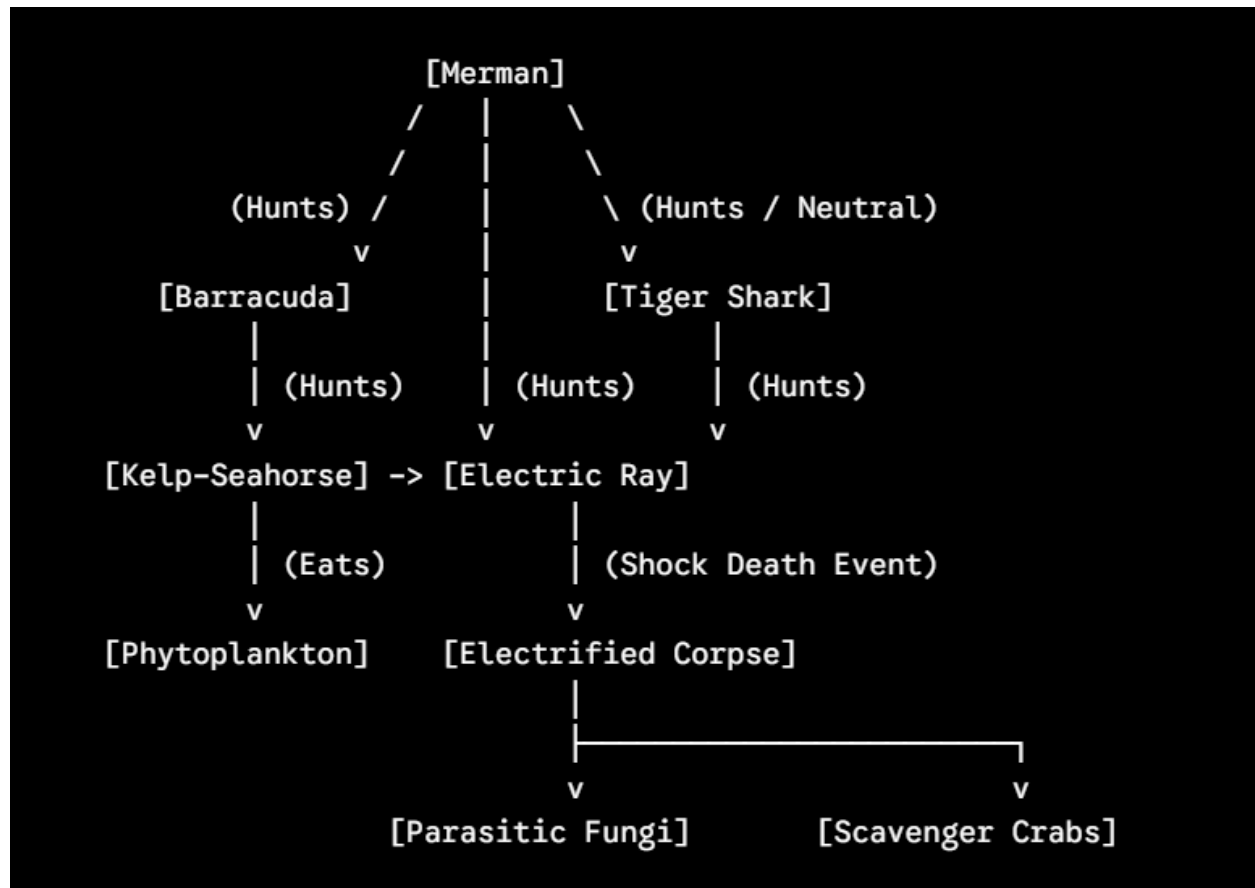
(Drawn by Blood Field) (Infects Surrounding Host)

2.1 Graph Interaction Network Mathematics The desire/weight $W(i, j)$ for Entity i to interact with Entity j is calculated dynamically per tick: $W(i, j) = H_i \cdot \Big(S(i, j) \cdot \text{TrophicMultiplier}(i, j) - \text{Toxicity}(j) - \text{Risk}(i, j) \Big) \cdot \text{Scent}(j)$ Where: * $H_i \in [0, 1]$: Hunger level of Entity i . * $S(i, j)$: Size suitability ratio. High score when $Mass_j \in [0.05 \cdot Mass_i, 0.8 \cdot Mass_i]$. * $Toxicity(j)$: Chemical danger (e.g., Electric Ray charge level, Poison Dart Frog skin, Spore infection). * $Risk(i, j)$: Danger of target's counter-attack, scaled by Entity i 's Bravery trait. * $Scent(j)$: Scent cloud intensity at j 's location (blood, decay, pheromones). ### 2.2 Entity State Transitions & Food Web Matrix

```

cpp
enum class EntityState { Alive, Stunned, Infected, Corpse, Skeletons, Consumed };
enum class InteractionType { Hunt, Scavenge, Parasitize, Symbiosis, Flee, AvoidZone };
struct FoodWebEdge {
    uint32_t targetSpeciesID;
    InteractionType type;
    float basePreferenceScore; // Base weight before dynamic modifiers
    bool triggersCorpseEvent; // Spawns secondary hazards on kill
};
  
```

Interconnected Ecosystem Matrix (Sample Ecosystem: The Weeds / Atlantis)



1. Merman Interactions:

- Hunts Barracudas and Kelp-Seahorses.
- Attacks Tiger Sharks in groups, but flees if isolated.
- Spears Electric Rays **Chain Event:** Spear conducts electric discharge $\rightarrow$ Merman is stunned/killed $\rightarrow$ Spawns **Merman Corpse** + **Electric Arc Area**.

2. Corpse & Scavenger Cascades:

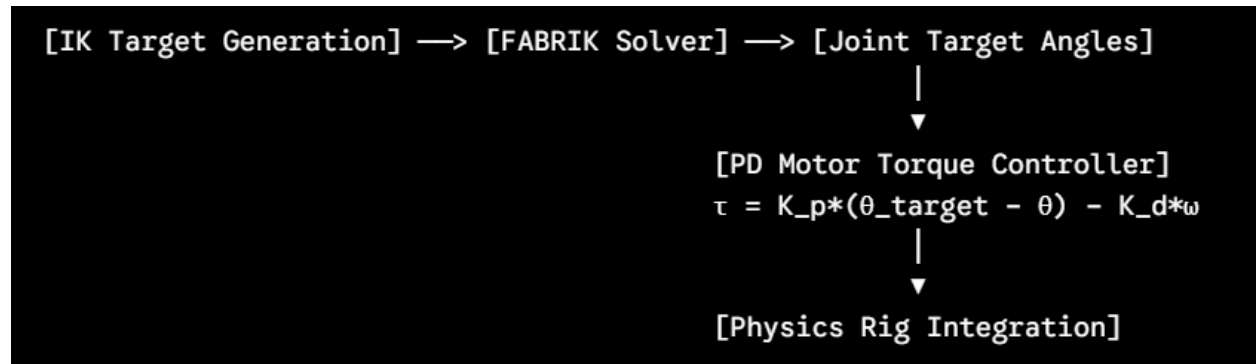
- **Merman Corpse** releases a expanding **Blood Cloud Field** into the Eulerian scent grid.
- Tiger Sharks within 50 meters drop current tasks to enter **Scavenge Mode**, converging on the corpse.
- Fungal Parasites infect the corpse; if left unconsumed for 10 seconds, it bursts into a **Spore Cloud** that infects nearby living Barracudas or Mermen, turning them hostiles.

3. Electric Ray Death Cascade:

- When killed by any predator, the Ray discharges its internal battery: 3-meter radial electric shock.
- Surrounding small prey (Seahorses, Phytoplankton) die instantly, creating a floating debris pile that attracts Scavenger Crabs from the sea floor.

3. PROCEDURAL ANIMATION & ACTIVE RAGDOLL IK ENGINE

No frame-by-frame sprites are used. Creatures are animated using physics joints, active motor torques, and Inverse Kinematics (FABRIK).



3.1 FABRIK Implementation with Joint Angle Constraints

```
struct BoneJoint {
    Vector2D position;
    float length;
    float minAngle; // Relative to parent joint
    float maxAngle;
};

class ConstrainedFABRIKSolver {
public:
    static void SolveChain(std::vector<BoneJoint>& chain, Vector2D target, float tolerance) {
        size_t n = chain.size();
        if (n < 2) return;

        // 1. Check reachability
        float totalLength = 0.0f;
        for (size_t i = 0; i < n - 1; ++i) totalLength += chain[i].length;

        Vector2D root = chain[0].position;
        if (Distance(root, target) >= totalLength) {
            for (size_t i = 0; i < n - 1; ++i) {
                Vector2D dir = Normalize(target - chain[i].position);
                chain[i + 1].position = chain[i].position + dir * chain[i].length;
            }
            return;
        }
    }
};
```

```

// 2. Iterative FABRIK Pass
int iterations = 0;
while (Distance(chain[n - 1].position, target) > tolerance && iterations < 8) {
    // Backward Pass
    chain[n - 1].position = target;
    for (int i = static_cast<int>(n) - 2; i >= 0; --i) {
        Vector2D dir = Normalize(chain[i].position - chain[i + 1].position);
        chain[i].position = chain[i + 1].position + dir * chain[i].length;
    }

    // Forward Pass with Angular Constraints
    chain[0].position = root;
    for (size_t i = 0; i < n - 1; ++i) {
        Vector2D dir = Normalize(chain[i + 1].position - chain[i].position);

        // Apply Angular Constraint relative to parent segment angle
        if (i > 0) {
            Vector2D parentDir = Normalize(chain[i].position - chain[i - 1].position);
            dir = ConstrainAngle(parentDir, dir, chain[i].minAngle, chain[i].maxAngle);
        }

        chain[i + 1].position = chain[i].position + dir * chain[i].length;
    }
    iterations++;
}

private:
static Vector2D ConstrainAngle(Vector2D parentDir, Vector2D currentDir, float minA, float
maxA) {
    float parentAngle = atan2(parentDir.y, parentDir.x);
    float currentAngle = atan2(currentDir.y, currentDir.x);
    float diff = currentAngle - parentAngle;

    // Wrap to [-PI, PI]
    while (diff < -3.14159f) diff += 6.28318f;
    while (diff > 3.14159f) diff -= 6.28318f;

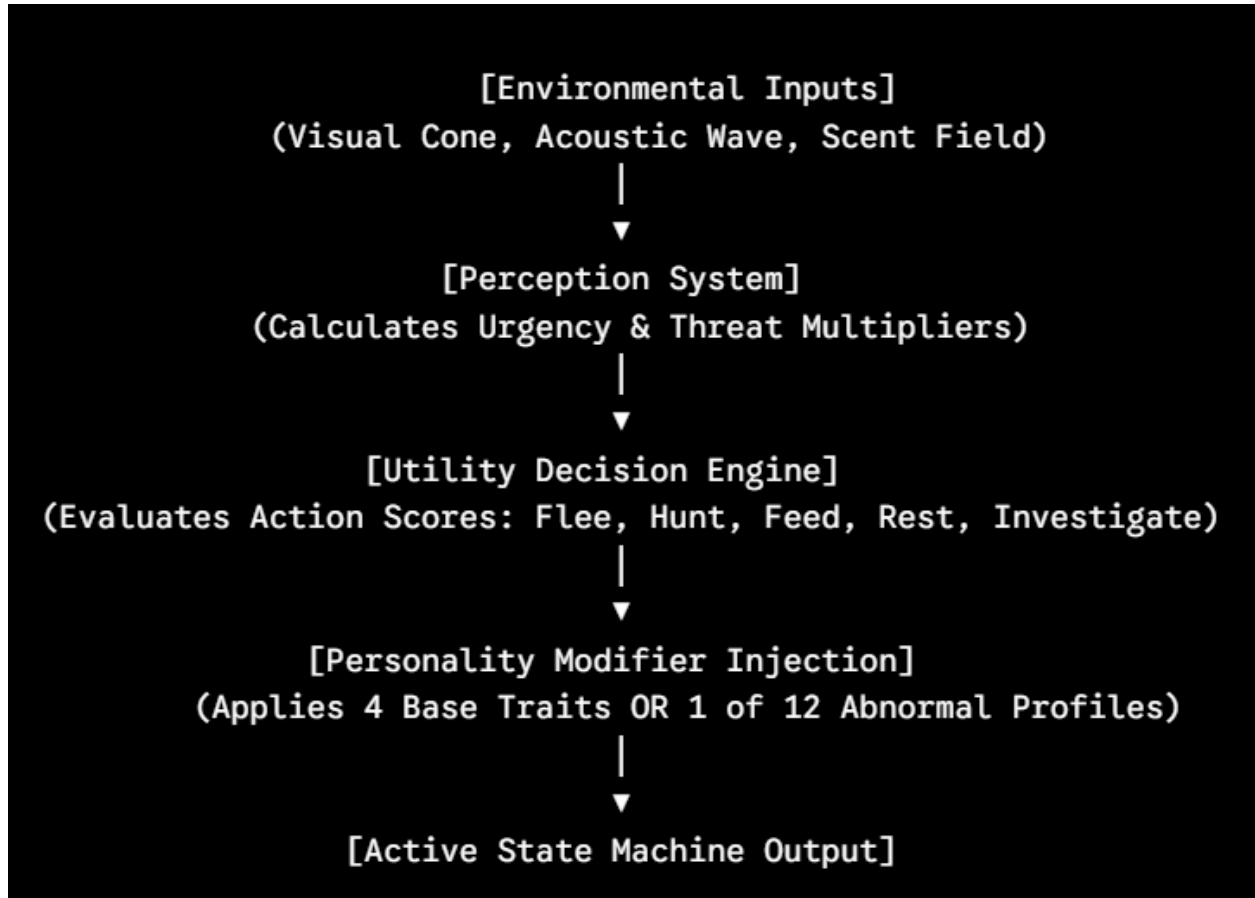
    // Clamp angle
    float clampedDiff = std::clamp(diff, minA, maxA);
    float finalAngle = parentAngle + clampedDiff;
    return { cos(finalAngle), sin(finalAngle) };
}

```

};

4. ADVANCED PERCEPTION & UTILITY AI ENGINE

Creatures decide actions using a continuous **Utility Evaluator** weighted by perception fields and personality traits.



4.1 Scent Grid Diffusion Engine (Eulerian Grid)

Scent, blood, and spores diffuse through a downsampled $W \times H$ scalar grid using Discrete Diffusion Equations:

$$C_{x,y}^{t+1} = C_{x,y}^t + \Big(D \cdot \nabla^2 C_{x,y}^t - k_{\text{decay}} \cdot C_{x,y}^t + \vec{v}_{\text{fluid}} \cdot \nabla C_{x,y}^t \Big) \Delta t$$

Entities evaluate scent gradient direction via central differencing:

$$\vec{G}_{scent} = \left(\frac{C_{x+1,y} - C_{x-1,y}}{2 \cdot \Delta x}, \frac{C_{x,y+1} - C_{x,y-1}}{2 \cdot \Delta y} \right)$$

4.2 Utility Action Calculation & The 12 Abnormal Personalities

The action score $U(A)$ for an available behavior $A \in \{Hunt, Flee, Scavenge, Rest, Investigate\}$ is computed as:

$$U(A) = \left(\sum_i w_i \cdot f_i(s) \right) \cdot \text{PersonalityModifier}(A)$$

```

struct PersonalityProfile {
    float aggression; // [0.0, 1.0]
    float bravery;    // [0.0, 1.0]
    float energy;     // [0.0, 1.0]
    float curiosity;  // [0.0, 1.0]
    AbnormalPersonality overrideType = AbnormalPersonality::None;
};

class UtilityEngine {
public:
    static float CalculateHuntUtility(const Entity& self, const Entity& target, const
    PersonalityProfile& p) {
        float hunger = self.hungerLevel; // [0.0 = Satiated, 1.0 = Starving]
        float sizeRatio = target.mass / self.mass;
        float distance = Distance(self.position, target.position);

        if (sizeRatio > 1.2f && p.bravery < 0.6f) return 0.0f; // Won't attack larger prey unless brave

        float distanceScore = 1.0f - (distance / self.perceptionRadius);
        float baseUtility = (hunger * 0.5f) + (distanceScore * 0.3f) + (p.aggression * 0.2f);

        // Apply Abnormal Overrides
        switch (p.overrideType) {
            case AbnormalPersonality::RabidEnraged:
                return 1.0f; // Always max hunt utility regardless of size/hunger
            case AbnormalPersonality::GluttonousDevourer:
                return baseUtility * 2.5f;
            case AbnormalPersonality::Kleptomaniac:
                return 0.0f; // Replaces hunting with item stealing
            default:
                return std::clamp(baseUtility, 0.0f, 1.0f);
        }
    }
};

```

```
};
```

Detailed Behavioral Execution Specification

```
class BehaviorStateMachine {
public:
    static void UpdateEntityBehavior(Entity& entity, EnvironmentContext& ctx) {
        // 1. Process Senses
        PerceptionData perception = SensorySystem::Process(entity, ctx);

        // 2. Evaluate Abnormal Overrides
        if (entity.personality.overrideType != AbnormalPersonality::None) {
            ExecuteAbnormalBehavior(entity, perception, ctx);
            return;
        }

        // 3. Standard Utility Evaluation
        float uHunt = UtilityEngine::CalculateHuntUtility(entity, perception.bestPrey,
entity.personality);
        float uFlee = UtilityEngine::CalculateFleeUtility(entity, perception.biggestThreat,
entity.personality);
        float uInvestigate = UtilityEngine::CalculateInvestigateUtility(entity,
perception.loudestSound, entity.personality);

        // 4. Select Dominant Utility Action
        if (uFlee > uHunt && uFlee > 0.5f) {
            entity.SetState(ActionState::Fleeing, perception.biggestThreat.position);
        } else if (uHunt > 0.4f) {
            entity.SetState(ActionState::Hunting, perception.bestPrey.position);
        } else if (uInvestigate > 0.3f) {
            entity.SetState(ActionState::Investigating, perception.loudestSound.position);
        } else {
            entity.SetState(ActionState::Wandering);
        }
    }

private:
    static void ExecuteAbnormalBehavior(Entity& e, const PerceptionData& p,
EnvironmentContext& ctx) {
        switch (e.personality.overrideType) {
            case AbnormalPersonality::RabidEnraged:
                // Attacks nearest target regardless of species, suffers continuous health loss
                e.health -= ctx.deltaTime * 1.5f;
        }
    }
};
```

```
e.targetPos = p.nearestLivingEntity.position;
e.SetState(ActionState::Attacking);
break;
```

```
case AbnormalPersonality::Kleptomaniac:
```

```
// Tracks player's loose relics or inventory items, grabs, and flees to level boundary
```

```
if (ctx.playerHasRelic) {
    e.targetPos = ctx.playerPosition;
    if (Distance(e.position, ctx.playerPosition) < 1.0f) {
        ctx.StealRelic();
        e.personality.overrideType = AbnormalPersonality::None; // Shift to standard flee
        e.SetState(ActionState::Fleeing);
    }
}
break;
```

```
case AbnormalPersonality::HyperProtectiveParent:
```

```
// Peaceful until player/predator approaches home Den (< 6.0m), then acquires
hyper-speed
```

```
if (Distance(ctx.playerPosition, e.homeDenPosition) < 6.0f) {
    e.movementForceMultiplier = 2.2f;
    e.targetPos = ctx.playerPosition;
    e.SetState(ActionState::Attacking);
} else {
    e.targetPos = e.homeDenPosition;
    e.SetState(ActionState::Guarding);
}
break;
```

```
case AbnormalPersonality::SuicidalSelfDestructive:
```

```
// Seeks explosive triggers (gunpowder barrels, electrical arcs) near targets
```

```
if (ctx.HasNearbyExplosive()) {
    e.targetPos = ctx.GetNearestExplosivePos();
    if (Distance(e.position, e.targetPos) < 0.8f) {
        ctx.TriggerExplosion(e.position);
    }
}
break;
```

```
case AbnormalPersonality::ParasiticHost:
```

```
// Latches onto larger entities or player, injecting control vectors into their inputs
```

```
if (!e.isLatched) {
    e.targetPos = p.biggestThreat.position;
    if (Distance(e.position, e.targetPos) < 0.5f) {
```

```

        e.LatchToTarget(p.biggestThreat);
    }
}
break;

case AbnormalPersonality::CovetousHoarder:
    // Collects loose items and drags them to its nest
    if (!e.isCarryingItem) {
        e.targetPos = ctx.GetNearestLooseItemPos();
    } else {
        e.targetPos = e.homeDenPosition;
    }
    break;

case AbnormalPersonality::SymbioticCompanion:
    // Emits light/speed boost aura and follows player closely
    e.targetPos = ctx.playerPosition + Vector2D{-1.5f, -1.0f};
    ctx.ApplySpeedAura(e.position, 4.0f);
    break;

case AbnormalPersonality::TerritorialTyrant:
    // Dominates a 12m radius zone. Attacks ANY entity entering
    for (auto& target : ctx.GetEntitiesInRadius(e.territoryCenter, 12.0f)) {
        if (target.id != e.id) {
            e.targetPos = target.position;
            e.SetState(ActionState::Attacking);
            return;
        }
    }
    break;

case AbnormalPersonality::PyromaniacShockSeeker:
    // Drawn directly to electrical arcs, light pulses, or powder trails
    e.targetPos = ctx.GetLoudestEnergySourcePos();
    break;

case AbnormalPersonality::GluttonousDevourer:
    // Consumes anything smaller than itself, gaining 5% mass per consumption
    if (p.bestPrey.mass < e.mass) {
        e.targetPos = p.bestPrey.position;
        if (Distance(e.position, e.targetPos) < 0.8f) {
            e.Consume(p.bestPrey);
            e.mass *= 1.05f;
            e.scale *= 1.03f;
        }
    }

```

```

    }
}
break;

case AbnormalPersonality::PhobicNyctophobic:
    // Panics when ambient light < 0.3, fleeing toward nearest bioluminescent source
    if (ctx.GetLightLevel(e.position) < 0.3f) {
        e.targetPos = ctx.GetNearestLightSourcePos();
        e.movementForceMultiplier = 1.8f;
        e.SetState(ActionState::Fleeing);
    }
    break;

case AbnormalPersonality::MalignantAlpha:
    // Emits continuous command wave, forcing all same-species within 15m to join its
    target attack
    e.targetPos = ctx.playerPosition;
    ctx.ForceSwarmTarget(e.speciesID, e.position, 15.0f, ctx.playerPosition);
    break;
}
}
};

```

5. VISUAL ART DIRECTION & ENVIRONMENT RENDERING SPECIFICATION

CRITICAL ARCHITECTURAL DIRECTIVE FOR CLAUDE CODE:

NEVER generate plain geometric boxes, monochrome placeholder rectangles, or untextured debug polygons for level terrain, entities, or props.

All level geometry must be rendered using **Auto-Tiled Textures**, **Normal Maps**, **Custom Shaders**, **Multi-Layer Parallax Scrolling**, and **Volumetric Particle Systems**.

[Render Pipeline Core] | —> Layer 0: Deep Parallax Background (0.2x speed, Vantablack + Fog Shader) | —> Layer 1: Midground Background Structures (0.5x speed, Silhouettes) | —> Layer 2: Main Playfield Terrain (1.0x speed, Dual-Grid Bitmask Auto-Tiling) | —> Layer 3: Dynamic Entities & IK Rigs (Normal-Mapped, Light Reactive) | —> Layer 4: Foreground Traversal Flora/Pipes (1.2x speed, Out-of-Focus Blur) | —> Shader Pass: Post-Processing (Caustics, Distortion, Bioluminescent Bloom)

5.1 Auto-Tiling System (Dual-Grid / Bitmask 8-Bit Rule)

Level terrain surfaces are rendered by evaluating surrounding cell states using an 8-bit bitmask rule to select correct decorative edge tiles (cracked metal, barnacle crusts, overgrown moss, carved Atlantean marble).

```
uint8_t ComputeTileBitmask(const Grid& grid, int x, int y) {
    uint8_t mask = 0;
    if (grid.IsSolid(x, y - 1)) mask |= 1; // North
    if (grid.IsSolid(x + 1, y - 1)) mask |= 2; // North-East
    if (grid.IsSolid(x + 1, y)) mask |= 4; // East
    if (grid.IsSolid(x + 1, y + 1)) mask |= 8; // South-East
    if (grid.IsSolid(x, y + 1)) mask |= 16; // South
    if (grid.IsSolid(x - 1, y + 1)) mask |= 32; // South-West
    if (grid.IsSolid(x - 1, y)) mask |= 64; // West
    if (grid.IsSolid(x - 1, y - 1)) mask |= 128; // North-West
    return mask;
}
```

5.2 Biome Visual Identity & Palette Specifications

Biome	Color Palette (Hex)	Primary Tile Textures	Visual Shader Effects	Environmental Props
1. The Pipes	#1A1C23 (Cast Iron) #D96B27 (Rust) #4E8752 (Slime)	Riveted iron plates, rusted pipe elbows, brass steam valves	Heat shimmer distortion, dripping condensation particles	Pressure gauges, escaping steam vents, hanging chains

2. The Hull	#0F2027 (Deep Ocean) #2C5364 (Submerged Steel) #E5A93C (Barnacles)	Barnacle-encrusted metal, rotting copper plating, kelp roots	Water caustic overlay shader, floating plankton particles	Broken ship framing, submerged anchors, hanging ropes
3. The Pirate Ship	#2C1D11 (Aged Oak) #C0A060 (Rope Canvas) #8B0000 (Crimson Sails)	Dark oak deck planks, weathered wood grain, brass trim	Dynamic lighting, sway distortion on rigging elements	Canvas sails, cannon ports, wooden barrels, ship wheel

4. The Island	#0B2B11 (Jungle Canopy) #5C3D2E (Mud/Bark) #00F0FF (Tropical Water)	Muddy rock faces, tropical vine overlays, mossy wood	Sun shaft volumetric god-rays, falling leaf particles	Tribal totems, rope bridges, coconut clusters, bamboo spikes
5. The Cave	#0D0E15 (Basalt) #2A3B50 (Moist Slate) #00FF88 (Biolum Fungus)	Wet basalt, jagged stalactite tiles, glowing spore veins	Echolocation sound-ripple wireframe overlay	Hanging stalactites, glowing mushroom clusters, stone arches

6. The Weeds	#051A12 (Abyssal Green) #1A5E3A (Kelp Fronds) #FF0055 (Coral Hazards)	Layered kelp stalks, brain coral platforms, sponge walls	Multi-layer fluid sway shader, depth fog vignette	Swaying giant kelp, floating spore pods, sea anemone clusters
7. Atlantis	#112233 (Sunken Basalt) #44AA99 (Patina Copper) #FFD700 (Glowing Runes)	Carved marble blocks, golden runic inlay tiles, aqueduct channels	Runic glow pulsing shader, dynamic water refraction	Sunken statues, clockwork gear mechanisms, broken arches

8. The Open Abyss	#020208 (Void Black)	Translucent Glass Sponges, slimy sheer trench walls	Edge pressure vignette, darkness sound-as-light shader	Black smoker vents, brine pool shelves, Siphonophore webs
	#00E5FF (Cyan Glass)			
	#A000FF (Abyssal Magenta)			

5.3 GLSL Water Caustics & Fluid Distortion Shader (**water_caustics.frag**)

```
#version 330 core
in vec2 FragTexCoord;
out vec4 FragColor;

uniform sampler2D sceneTexture;
uniform float time;
uniform vec2 waveStrength;

void main() {
    vec2 coord = FragTexCoord;

    // Calculate sine-wave offset grid for underwater distortion
    float distortionX = sin(coord.y * 20.0 + time * 3.0) * 0.003;
    float distortionY = cos(coord.x * 15.0 + time * 2.5) * 0.003;
    coord += vec2(distortionX, distortionY);

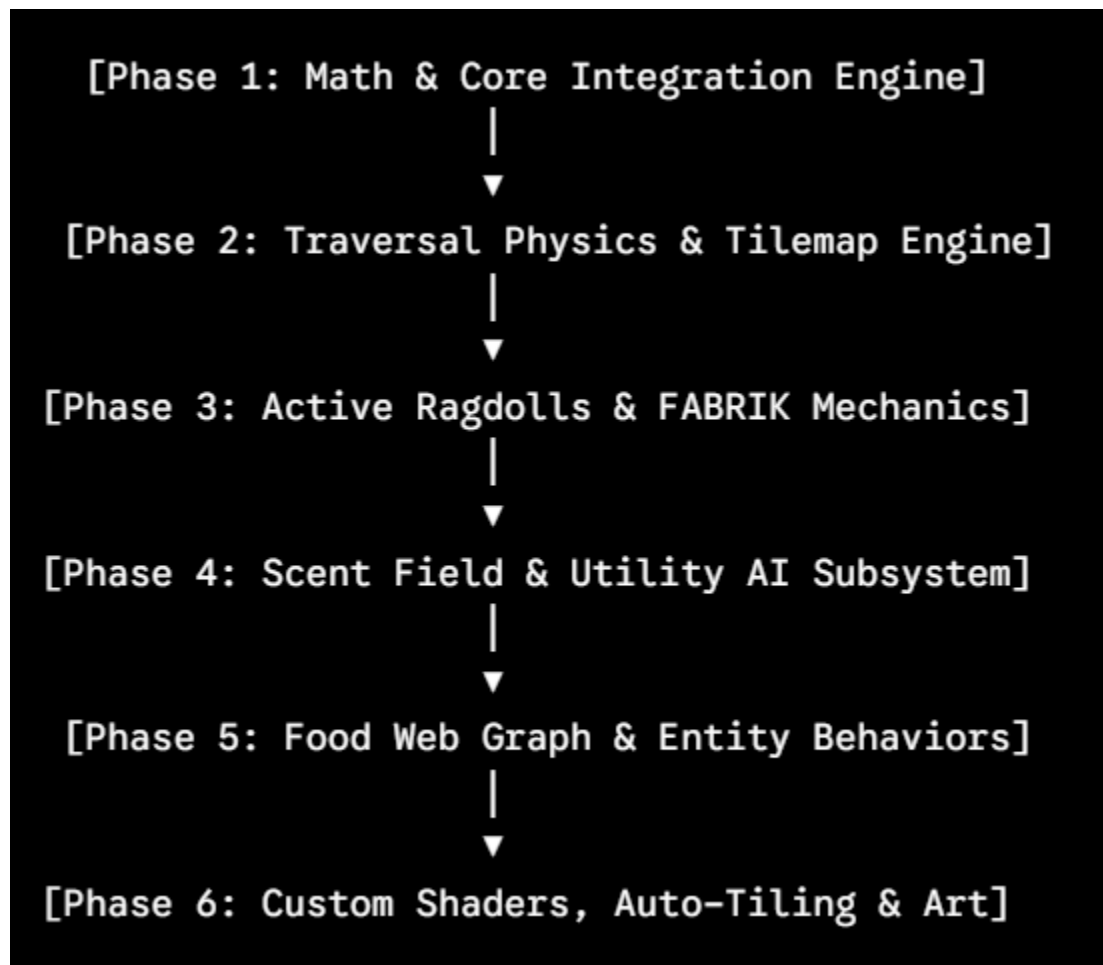
    // Dynamic Caustic Pattern
    float caustic = sin(coord.x * 40.0 + coord.y * 40.0 + time * 4.0) *
        cos(coord.x * 20.0 - coord.y * 30.0 + time * 2.0);
    caustic = std::clamp(pow(caustic + 0.8, 3.0), 0.0, 1.0) * 0.15;

    vec4 sceneColor = texture(sceneTexture, coord);
    vec3 waterTint = vec3(0.05, 0.25, 0.35); // Abyssal teal tint
```

```
FragColor = vec4(mix(sceneColor.rgb, waterTint, 0.25) + vec3(caustic), sceneColor.a);  
}
```

6. GRANULAR IMPLEMENTATION ROADMAP FOR CLAUDE CODE (OPUS 5.5 IN C++)

Claude Code must implement this architecture in **discrete, non-destructive technical phases**.



Phase 1: Core Physics & Traversal Mechanics

- **Step 1.1:** Write basic math types ([Vector2D](#), [Matrix2x2](#), [Ray2D](#)) and velocity-verlet integrator functions.
- **Step 1.2:** Implement [PlayerPhysicsController](#) with full support for state transitions: Ground Sprint, Ground Slide, Impact Roll, Hydro-Glide, Water Dash, and Parachute Brake.

- **Step 1.3:** Build narrowphase collision solver handling tile map collision manifolds and variable friction materials.

Phase 2: Procedural Tilemap & Generator

- **Step 2.1:** Implement [CellularAutomata](#) generator for Cave/Pipes biomes.
- **Step 2.2:** Build vertical streaming chunk engine for *The Open Abyss* with sliding generation windows.
- **Step 2.3:** Write Poisson-Disc Den spawner placement system.

Phase 3: IK Rigging & Procedural Locomotion

- **Step 3.1:** Implement [ConstrainedFABRIKSolver](#) with parent-relative angle limits.
- **Step 3.2:** Write locomotion algorithms: Serpentine Sine Wave, Multipod Tripod Gait, Hydrodynamic Swimming Forces, and Biped PD Motors.

Phase 4: Scent Diffusion Grid & Utility Perception Engine

- **Step 4.1:** Write downsampled $W \times H$ Eulerian grid class handling scent diffusion, decay, and gradient calculations.
- **Step 4.2:** Build [PerceptionSystem](#) computing raycast vision cones, acoustic wave intensity, and scent gradient detection.
- **Step 4.3:** Build [UtilityEngine](#) computing action preference scores based on traits ($U \in [0.0, 1.0]$).

Phase 5: Interconnected Food Web & Behavior Logic

- **Step 5.1:** Implement [FoodWebGraph](#) managing directed interaction edges, dynamic preference scores, and state transitions (Corpse, Stunned, Infected).
- **Step 5.2:** Program the 12 Abnormal Personality overrides within [BehaviorStateMachine](#).
- **Step 5.3:** Program procedural AI logic for all 80 distinct entities across the 8 biomes.

Phase 6: Shaders, Auto-Tiling, Parallax & Polish

- **Step 6.1:** Implement 8-bit dual-grid bitmask auto-tiling system for terrain surfaces.
- **Step 6.2:** Integrate GLSL custom shaders ([water_caustics.frag](#), [bioluminescence.frag](#), [pressure_vignette.frag](#)).
- **Step 6.3:** Build multi-layer parallax rendering engine with depth fog.

7. ARCHITECTURAL QUESTIONS & PROPOSALS FOR EXPANSION

Before finalizing code generation for Phase 1, consider these architectural decisions:

1. **Corpse Rot Mechanics & Disease:** Should rotting corpses left unconsumed in water gradually degrade overall biome water visibility (adding a dynamic darkness haze shader) or stay localized to scent clouds?

2. **Entity Limb Severance / Active Ragdoll Dismemberment:** When high-mass predators (e.g., Tiger Sharks, Abyssal Leviathans) bite smaller targets, should the FABRIK IK chain sever into separate active rigid-body pieces that retain independent scent emission and attraction?
3. **Player Customization & Relic Mechanics:** Should relics collected in high-difficulty areas (like *The Pirate Ship* or *Atlantis*) grant passive physics overrides to the player (e.g., permanent reduction in fluid drag, immunity to electric arcs, or scent-masking abilities)?

With prompt: develop everything further

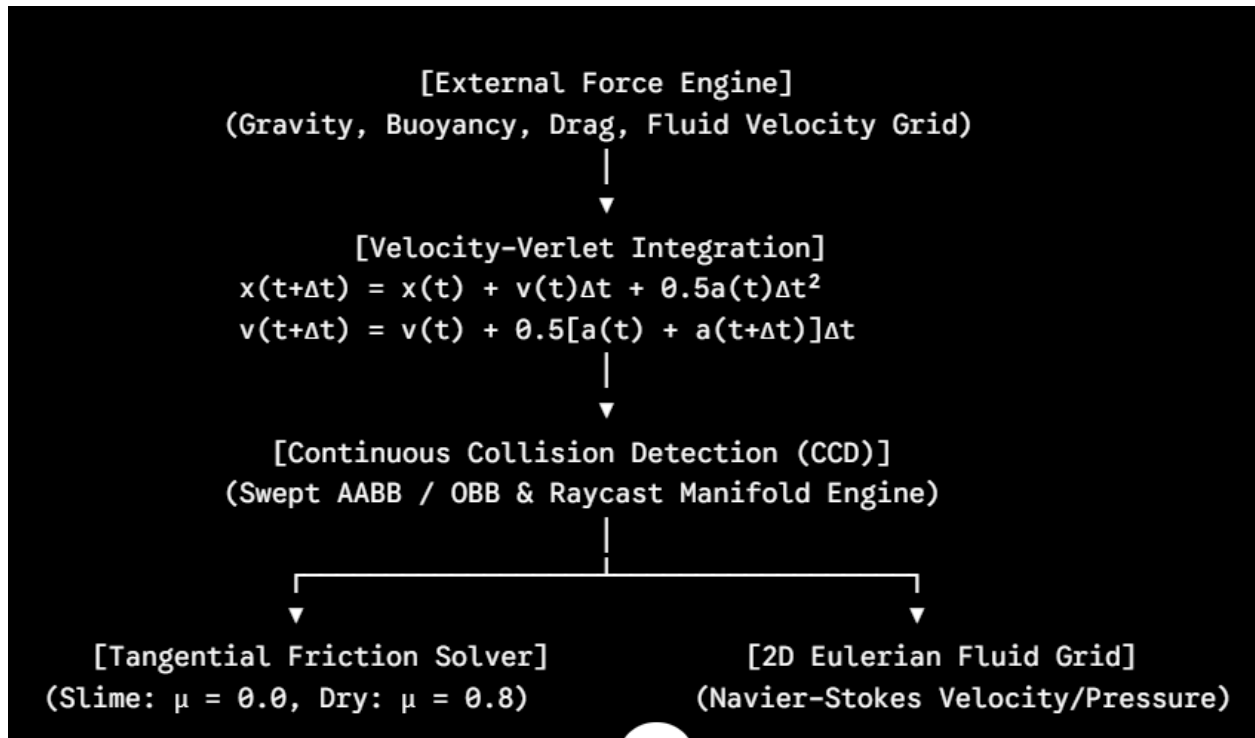
DIRECTIONS1: STANDALONE PARKOUR ENGINE ARCHITECTURE & SPECIFICATION

Target System: Production C++17 / C++20 (SFML 2.6+ or SDL2 / OpenGL 3.3+ Core)

LLM Execution Directives: Read as strict technical specifications. Code generation must follow the modular phase breakdown in Section 7. Do not substitute simple geometric shapes or untextured colored boxes for finished gameplay assets, dynamic terrain tiles, active ragdolls, or shader passes.

1. ADVANCED ENGINE PHYSICS & EULERIAN FLUID SIMULATION

The physics subsystem utilizes an **Impulse-Based Velocity-Verlet Integrator** coupled with a 2D Eulerian fluid velocity grid. This enables momentum preservation during transitions between land parkour and 3D hydrodynamic traversal.



1.1 Mathematical Mechanics Equations

Fluid Drag & Buoyancy

Entities calculate net force vectors using:

$$F_{net} = F_{gravity} + F_{buoyancy} + F_{drag} + F_{thrust}$$

Where:

- **Gravity:** $F_{gravity} = m \cdot g \cdot \hat{j}$
- **Hydrodynamic Buoyancy:** $F_{buoyancy} = -\rho_{fluid} \cdot V_{submerged} \cdot g \cdot \hat{j}$
 - Standard Water Density: $\rho_{water} = 1000kg/m^3$
 - Abyssal Brine Density: $\rho_{brine} = 2400kg/m^3$
- **Quadratic Fluid Drag:**

$$F_{\text{drag}} = -\frac{1}{2} \cdot \rho_{\text{fluid}} \cdot C_D \cdot A \cdot |\vec{v}_{\text{rel}}| \cdot \vec{v}_{\text{rel}}$$

Where $|\vec{v}_{\text{rel}}| = |\vec{v}_{\text{entity}} - \vec{v}_{\text{fluid_grid}}|$, C_D is the fluid drag coefficient, and A is the frontal cross-sectional area.

1.2 C++ Physics Core & Traversal Controller ([PhysicsEngine.hpp](#))

```
#pragma once
#include <SFML/System/Vector2.hpp>
#include <vector>
#include <algorithm>
#include <cmath>
```

```
enum class SurfaceMaterial { DryRock, Metal, AbyssalSlime, Web, GlassSponge, BrineMucous };
enum class TraversalState { Sprint, Slide, ImpactRoll, HydroGlide, WaterDash, ParachuteBrake };
};
```

```
struct MaterialProperties {
    float friction = 0.8f;
    float elasticity = 0.2f;
    float brakeThreshold = 500.0f; // Force required to break material
    bool generatesSpores = false;
};
```

```
struct PhysicsBody {
```

```

sf::Vector2f position{0.0f, 0.0f};
sf::Vector2f velocity{0.0f, 0.0f};
sf::Vector2f acceleration{0.0f, 0.0f};
float mass = 70.0f;          // kg
float volume = 0.07f;        // m^3
float dragCoefficient = 0.1f;
float crossSectionArea = 0.5f; // m^2
bool isSubmerged = false;
TraversalState state = TraversalState::Sprint;
};

class PhysicsIntegrator {
public:
    static void Integrate(PhysicsBody& body, const sf::Vector2f& fluidVelocity, float fluidDensity,
float dt) {
        // 1. Calculate Relative Velocity
        sf::Vector2f relVel = body.velocity - fluidVelocity;
        float speed = std::sqrt(relVel.x * relVel.x + relVel.y * relVel.y);

        // 2. Gravitational & Buoyant Forces
        float g = 9.81f * 100.0f; // Scale factor: 1 meter = 100 pixels
        sf::Vector2f fGravity = {0.0f, body.mass * g};
        sf::Vector2f fBuoyancy = {0.0f, 0.0f};

        if (body.isSubmerged) {
            fBuoyancy.y = -(fluidDensity * body.volume * g);
        }

        // 3. Hydrodynamic Drag
        sf::Vector2f fDrag = {0.0f, 0.0f};
        if (speed > 0.0001f) {
            float dragMag = 0.5f * fluidDensity * body.dragCoefficient * body.crossSectionArea *
speed * speed;
            fDrag = -(relVel / speed) * dragMag;
        }

        // 4. Sum Accelerations
        sf::Vector2f netForce = fGravity + fBuoyancy + fDrag;
        sf::Vector2f newAccel = netForce / body.mass;

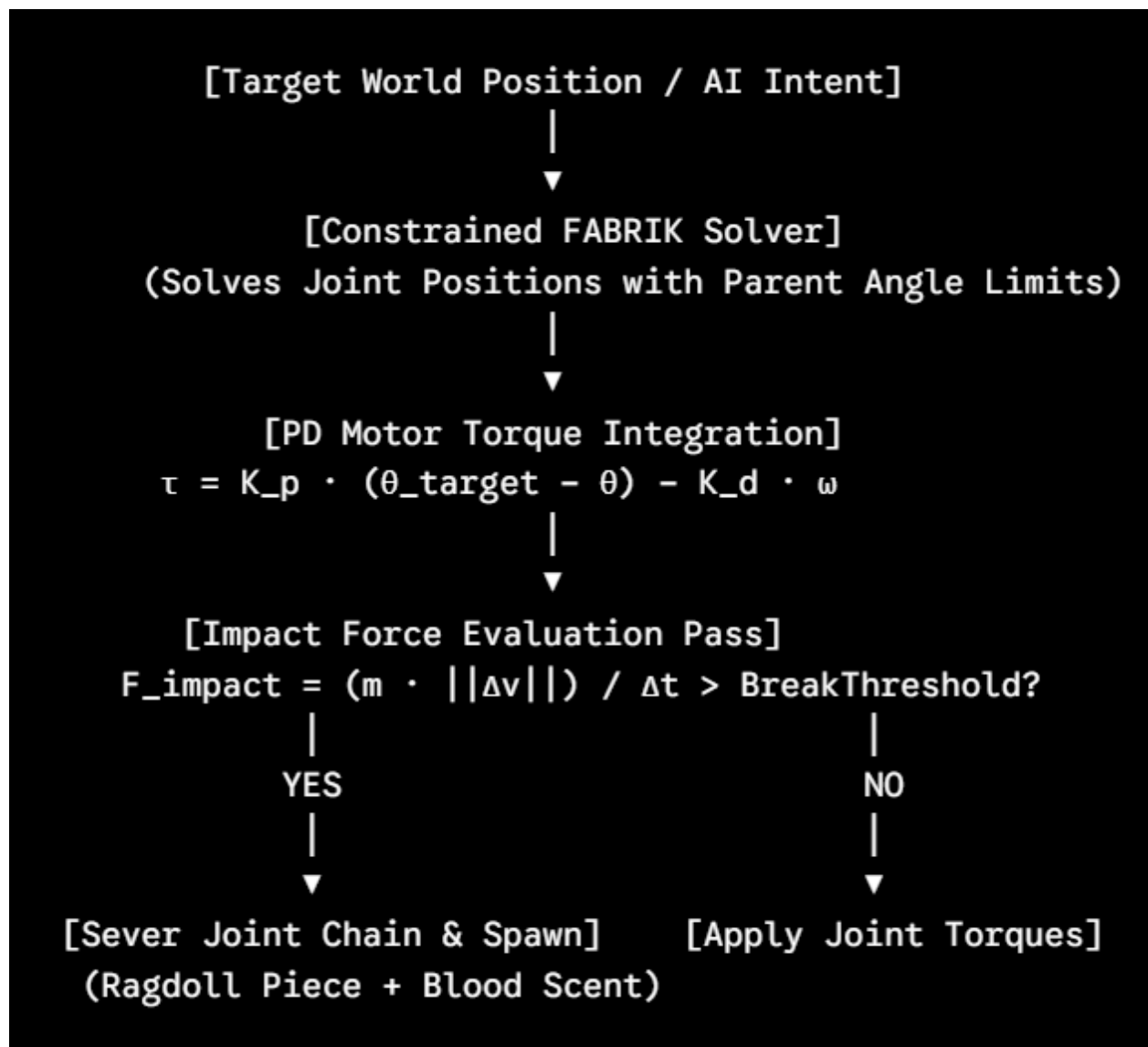
        // 5. Velocity-Verlet Integration
        body.position += body.velocity * dt + 0.5f * body.acceleration * (dt * dt);
        body.velocity += 0.5f * (body.acceleration + newAccel) * dt;
        body.acceleration = newAccel;
    }
};

```

```
}  
};
```

2. ACTIVE RAGDOLLS, IK DYNAMICS & DYNAMIC DISMEMBERMENT

Animation is physics-driven. Entities use **Active Ragdolls** controlled by Proportional-Derivative (PD) motor torques overlaid on a **Constrained FABRIK (Forward And Backward Reaching Inverse Kinematics)** solver.



2.1 Dynamic Limb Severance Engine (**RagdollSystem.hpp**)

When an entity receives an impact force $F_{\text{impact}} = \frac{m \cdot |\vec{v}|}{\Delta t}$ exceeding **BreakThreshold**, the joint chain severs. The severed limb transforms into an independent physical entity emitting blood scent into the Eulerian grid.

```
struct RagdollNode {
    sf::Vector2f position;
    sf::Vector2f velocity;
    float length;
    float minAngle; // Angle limits relative to parent
    float maxAngle;
    float currentAngle;
    float breakThreshold; // Force in Newtons causing limb severance
    bool isSevered = false;
    uint32_t parentIndex = 0;
};

class RagdollRig {
public:
    std::vector<RagdollNode> joints;

    void ProcessImpact(size_t jointIdx, float impactForce, std::vector<PhysicsBody>&
worldRigidBodies) {
        if (jointIdx >= joints.size()) return;

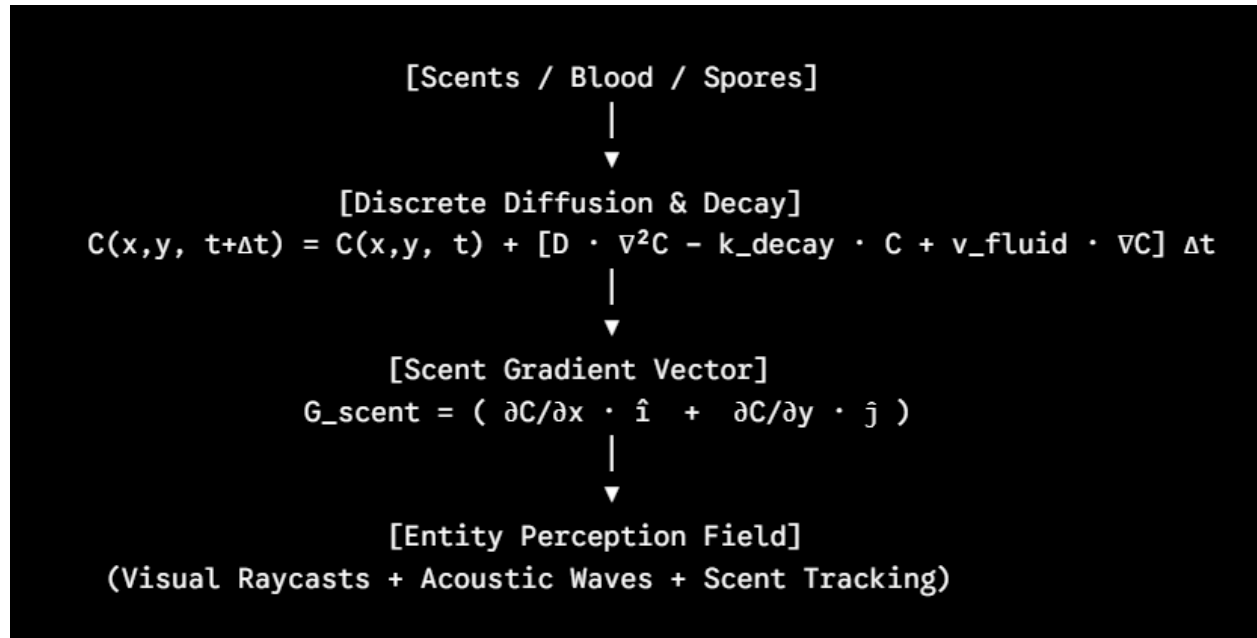
        if (impactForce > joints[jointIdx].breakThreshold && !joints[jointIdx].isSevered) {
            joints[jointIdx].isSevered = true;

            // Spawn detached piece as a new independent Physics Body
            PhysicsBody severedLimb;
            severedLimb.position = joints[jointIdx].position;
            severedLimb.velocity = joints[jointIdx].velocity + sf::Vector2f{(float)(rand()%100 - 50),
-150.0f};
            severedLimb.mass = 5.0f;
            severedLimb.volume = 0.005f;

            worldRigidBodies.push_back(severedLimb);
        }
    }
};
```

3. EULERIAN SCENT GRID & PERCEPTION ENGINE

Scent, blood, and spores propagate through a downsampled discrete scalar grid using Advection-Diffusion-Decay equations.



3.1 C++ Scent Grid Implementation ([ScentGrid.hpp](#))

```
#pragma once
#include <vector>
#include <cmath>
#include <SFML/System/Vector2.hpp>

class ScentGrid {
private:
    int width, height;
    float cellSize;
    float diffusionRate = 0.15f;
    float decayRate = 0.02f;
    std::vector<float> grid;
    std::vector<float> nextGrid;

public:
    ScentGrid(int w, int h, float scale) : width(w), height(h), cellSize(scale) {
        grid.resize(w * h, 0.0f);
        nextGrid.resize(w * h, 0.0f);
    }
};
```

```

void EmitScent(int x, int y, float amount) {
    if (x >= 0 && x < width && y >= 0 && y < height) {
        grid[y * width + x] += amount;
    }
}

void Update(float dt, const std::vector<sf::Vector2f>& fluidVelocities) {
    for (int y = 1; y < height - 1; ++y) {
        for (int x = 1; x < width - 1; ++x) {
            int idx = y * width + x;

            // Discrete Laplace Operator (Diffusion)
            float laplacian = grid[idx - 1] + grid[idx + 1] +
                grid[(y - 1) * width + x] + grid[(y + 1) * width + x] - 4.0f * grid[idx];

            // Fluid Advection Offset
            sf::Vector2f vel = fluidVelocities[idx];
            int advX = std::clamp(x - (int)(vel.x * dt / cellSize), 0, width - 1);
            int advY = std::clamp(y - (int)(vel.y * dt / cellSize), 0, height - 1);
            float advectedScent = grid[advY * width + advX];

            // Diffusion + Advection + Decay
            nextGrid[idx] = advectedScent + (diffusionRate * laplacian - decayRate *
advectedScent) * dt;
            nextGrid[idx] = std::max(0.0f, nextGrid[idx]);
        }
    }
    grid = nextGrid;
}

sf::Vector2f SampleGradient(float worldX, float worldY) const {
    int gx = std::clamp((int)(worldX / cellSize), 1, width - 2);
    int gy = std::clamp((int)(worldY / cellSize), 1, height - 2);

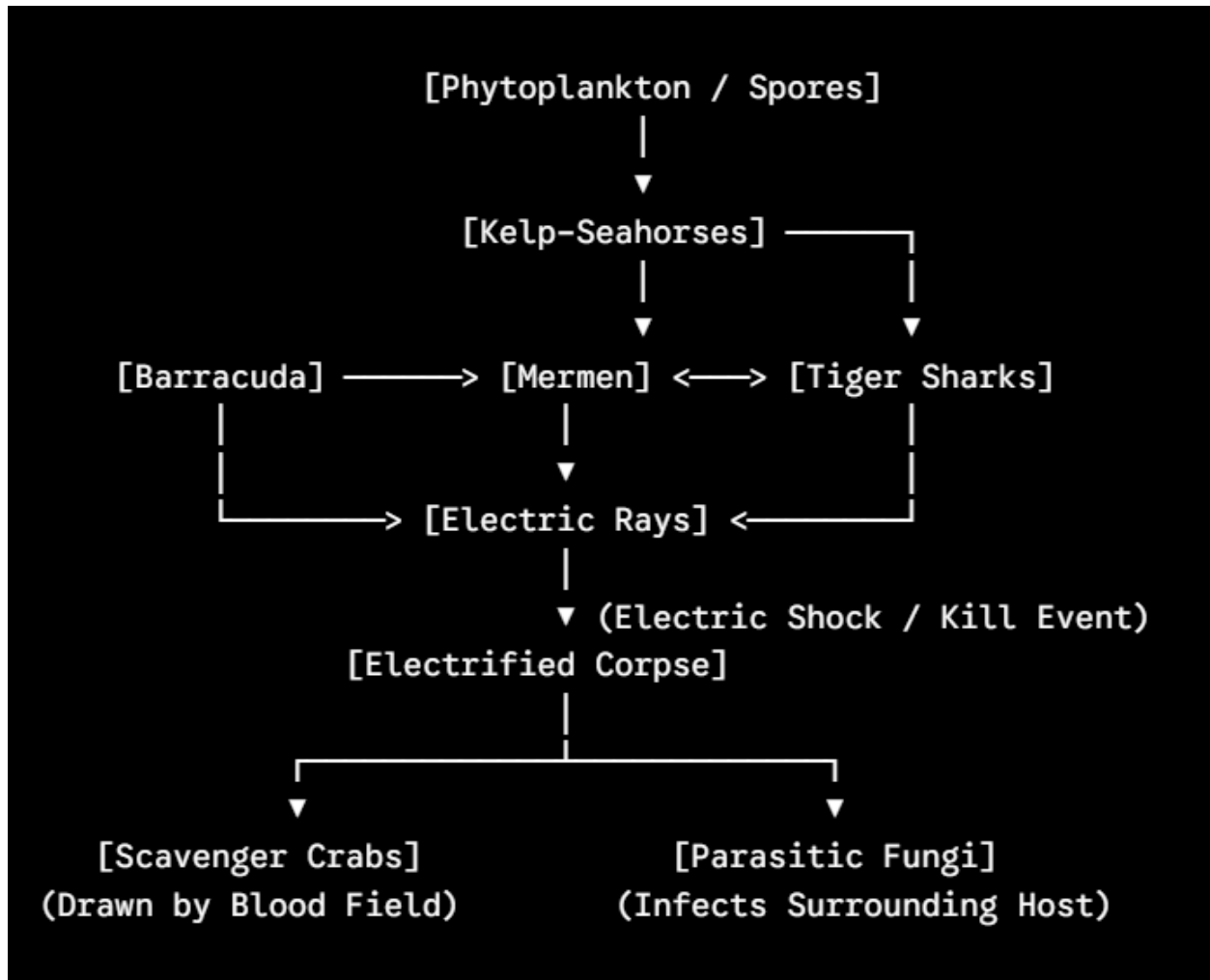
    float dx = (grid[gy * width + (gx + 1)] - grid[gy * width + (gx - 1)]) * 0.5f;
    float dy = (grid[(gy + 1) * width + gx] - grid[(gy - 1) * width + gx]) * 0.5f;

    return {dx, dy};
}
};

```

4. GRAPH-BASED FOOD WEB & ECOSYSTEM MATRIX ($G = (V, E)$)

Ecosystem interactions operate as an interconnected directed web $G = (V, E)$. Nodes V represent active entities, corpses, or severed limbs. Edges E represent interaction preference scores calculated dynamically per tick.



4.1 Ecosystem Interaction Chains Across All 8 Biomes

1. The Pipes (Micro-Ecosystem)

Design Rule: Insects and rodents only. All player hazards are indirect collateral from inter-species conflicts.

- **Chain:** Dust Moths swarm light leaks $\rightarrow$ Water-Spiders build webs across pipe bends to catch Moths $\rightarrow$ Centipedes climb webs to eat Moths $\rightarrow$ Blind Pipe-Rats bite rusted pipe walls to reach Centipedes $\rightarrow$ Rust-Mites burst from pipe breaches to consume metal flakes $\rightarrow$ Pillbugs curl into rolling balls when touched by Rust-Mites

\$\to\$ Scavenger Mice hunt Pillbugs but get bowled over by rolling shells \$\to\$ Cockroaches fight Mice for scraps \$\to\$ Stepped-on Glow-Beetles emit light pulses \$\to\$ Cave Crickets panic from light pulses, triggering stampedes over platform footholds.

2. The Hull (Submerged Outer Hull)

- **Chain:** Cleaner Shrimp groom Moray Eels in hull breaches \$\to\$ Octopuses camouflage against hull textures to ambush Shrimp \$\to\$ Barnacle Crabs pinch Octopuses that land too close \$\to\$ Octopuses spray ink clouds when pinched \$\to\$ Pufferfish panic inside ink clouds and inflate into spiked hazards \$\to\$ Hull-Leeches detach from inflated Pufferfish and float in currents \$\to\$ Stinging Anemones feed on floating Leeches \$\to\$ Hermit Crabs scavenge Anemone scraps \$\to\$ Brittle-Stars form fragile carpet mats over slick hull surfaces that break under high-speed player dashes.

3. The Pirate Ship (Deck & Rigging)

- **Chain:** Ship Rats scurry toward deck crates \$\to\$ Stray Cats stalk Rats along cannon ports \$\to\$ Parakeets screech when Cats or Players rush past \$\to\$ Short-Range Pirates attack toward screeching Parakeets \$\to\$ Long-Range Pirates fire flintlocks at Short-Range commotion \$\to\$ Gunpowder Monkeys grab dropped flintlock embers and drop powder trails \$\to\$ Flea Swarms scatter from dog coats during explosions \$\to\$ Infected Guard Dogs go berserk \$\to\$ Barn Owls snatch fleeing Monkeys off rigging \$\to\$ Albatrosses dive-bomb Owls to steal dropped loot.

4. The Island (Tribal Jungle Terrain)

- **Chain:** Tribal Warriors fire blowdarts at Boars \$\to\$ Enraged Boars charge through jungle brush \$\to\$ Canopy Snakes drop onto charging Boars \$\to\$ Monitor Lizards scavenge fallen targets \$\to\$ Fruit Bats swarm when charging Boars shake roost trees \$\to\$ Web-Spinning Spiders catch flying Bats in canopy webs (which double as trampolines for the player) \$\to\$ Coconut Crabs cut coconuts onto entities below \$\to\$ Poison Dart Frogs apply poison to entities stepping on them \$\to\$ Seagulls dive-bomb Crabs \$\to\$ Hunting Dogs track poisoned/bleeding targets.

5. The Cave (Subterranean Tunnels)

- **Chain:** Stalactite Spiders trigger Cave Bat roosts \$\to\$ Bats emit shriek frequencies that loosen stalactites \$\to\$ Falling stalactites crush Rock-Boring Urchins \$\to\$ Giant Tube Worm platforms retract from impact shockwaves \$\to\$ Mutated Crustaceans feed on crushed Urchins \$\to\$ Bioluminescent Jellies flash when hit by water turbulence \$\to\$ Pale Salamanders hunt Jellies using light flashes \$\to\$ Fungal Beetles explode into spore clouds when killed \$\to\$ Cave Leeches drop from ceilings onto spored targets.

6. The Weeds (Vertical Kelp Forest)

- **Chain:** Phytoplankton clouds obscure vision \$\to\$ Kelp-Seahorses filter-feed on Phytoplankton \$\to\$ Barracudas hunt Seahorses \$\to\$ Manatees swim through kelp as moving platforms \$\to\$ Tiger Sharks hunt Barracuda packs \$\to\$ Electric Rays emit shockwaves when bumped by Manatees \$\to\$ Mermen spear stunned Sharks \$\to\$

Fungal Parasites infect wounded Mermen, forcing enraged behavior \$to\$ Snapping Turtles feed on Parasites \$to\$ Sea Kraits target Turtle nests.

7. Atlantis (Crumbling Sunken Ruins)

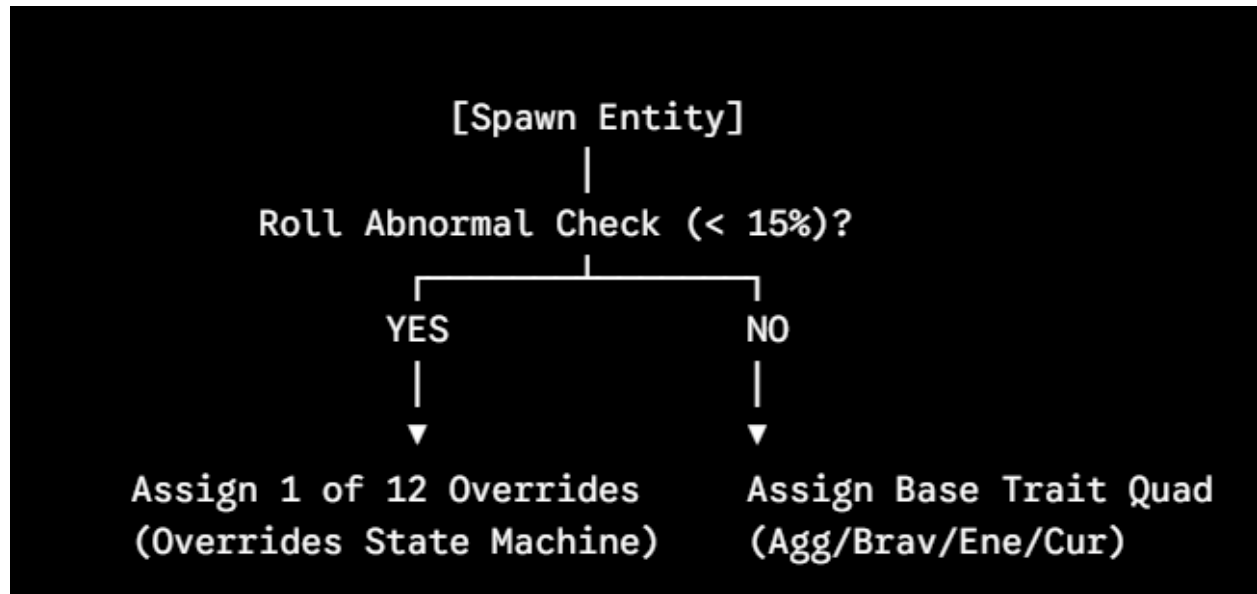
- **Chain:** Clockwork Drones fire energy pulses at biologicals \$to\$ Abyssal Sharks crush Drones, causing electrical discharges \$to\$ Stone-Crabs dislodge from ancient arches during discharges, triggering arch collapses \$to\$ Deep-Sea Anglerfish illuminate collapsed paths \$to\$ Ghostly Manta Rays flee light, creating high-speed Hydro-Glide slipstreams \$to\$ Living Marble Gargoyles awaken when slipstreams hit their pedestals \$to\$ The "Lost Ones" flee Gargoyles \$to\$ Lionfish block narrow corridors with poisonous spines \$to\$ Armored Nautiluses form force shields blocking Lionfish \$to\$ Ruins Octopuses scavenge Drone parts to build shelter.

8. The Open Abyss (Endless Vertical Descent)

- **Chain:** Abyssal Leviathans dive straight down, generating hydrodynamic down-drafts \$to\$ Giant Isopods get dislodged by down-drafts into high-velocity falling hazards \$to\$ Glass Sponges act as landing platforms but shatter under heavy impacts \$to\$ Deep-Sea Hatchetfish scatter from shattered Glass Sponges \$to\$ Gulper Eels snap at Hatchetfish based on acoustic vibrations \$to\$ Siphonophore Colonies snag lunging Eels, releasing toxins \$to\$ Bioluminescent Plankton illuminate upon contact with thrashing Eels \$to\$ Vampire Squid jet downward to escape illuminated zones \$to\$ Hadal Trench-Worms ambush fleeing Squid from wall crevices \$to\$ Heavy Brine-Slugs secrete slippery mucous in brine pools if Trench-Worms miss targets and strike brine ledges.

5. UTILITY AI ENGINE & THE 12 ABNORMAL PERSONALITIES

Creatures evaluate actions using a **Utility Decision Evaluator** modified by personality traits and abnormal behavior profiles.



5.1 C++ Utility Engine & Abnormal State Controller ([AIBehaviorEngine.hpp](#))

```
#pragma once
#include <SFML/System/Vector2.hpp>
#include <vector>
#include <cmath>
#include <algorithm>

enum class AbnormalPersonality {
    None, RabidEnraged, Kleptomaniac, HyperProtectiveParent, SuicidalSelfDestructive,
    ParasiticHost, CovetousHoarder, SymbioticCompanion, TerritorialTyrant,
    PyromaniacShockSeeker, GluttonousDevourer, PhobicNyctophobic, MalignantAlpha
};

struct EntityAIState {
    sf::Vector2f position;
    sf::Vector2f targetPosition;
    float health = 100.0f;
    float mass = 50.0f;
    float scale = 1.0f;
    float aggression = 0.5f;
    float bravery = 0.5f;
    bool isLatched = false;
    AbnormalPersonality abnormalType = AbnormalPersonality::None;
};

class AIBehaviorEngine {
```

```

public:
    static void UpdateAI(EntityAIState& entity, const sf::Vector2f& playerPos, float lightLevel, float
dt) {
        // Evaluate Abnormal Overrides
        if (entity.abnormalType != AbnormalPersonality::None) {
            ExecuteAbnormalProfile(entity, playerPos, lightLevel, dt);
            return;
        }

        // Standard Utility Steering towards target position
        sf::Vector2f dir = playerPos - entity.position;
        float dist = std::sqrt(dir.x * dir.x + dir.y * dir.y);
        if (dist > 0.001f) {
            entity.targetPosition = entity.position + (dir / dist) * 150.0f * dt;
        }
    }
}

```

```

private:
    static void ExecuteAbnormalProfile(EntityAIState& e, const sf::Vector2f& playerPos, float
lightLevel, float dt) {
        switch (e.abnormalType) {
            case AbnormalPersonality::RabidEnraged:
                e.health -= dt * 2.0f; // Continuous health decay
                e.targetPosition = playerPos; // Aggressively hunts player
                break;

            case AbnormalPersonality::GluttonousDevourer:
                e.mass *= (1.0f + 0.01f * dt);
                e.scale *= (1.0f + 0.005f * dt);
                e.targetPosition = playerPos;
                break;

            case AbnormalPersonality::PhobicNyctophobic:
                if (lightLevel < 0.3f) {
                    // Panic and sprint away from dark areas
                    e.targetPosition = e.position - (playerPos - e.position) * 2.0f;
                }
                break;

            case AbnormalPersonality::Kleptomaniac:
                e.targetPosition = playerPos; // Approaches player to steal items
                break;

            case AbnormalPersonality::SuicidalSelfDestructive:

```

```

        // Pathfinds directly into local hazardous entities or triggers
        e.targetPosition = playerPos;
        break;

    default:
        e.targetPosition = playerPos;
        break;
    }
}
};

```

6. VISUAL ART DIRECTION, AUTO-TILING & GLSL SHADERS

CRITICAL RENDER ARCHITECTURE DIRECTIVE FOR CLAUDE CODE:

NEVER generate plain geometric boxes, monochrome placeholder rectangles, or untextured debug polygons for level terrain, entities, or props.

Render all environments using **Dual-Grid Bitmask Auto-Tiling**, **Multi-Layer Parallax**, **Volumetric Particles**, and **Production GLSL Shaders**.

6.1 Dual-Grid / Bitmask 8-Bit Terrain Auto-Tiler

Terrain edges automatically evaluate neighbor cells using an 8-bit mask to select appropriate art tiles (cracked metal, barnacle crusts, overgrown moss, carved Atlantean marble).

```

uint8_t ComputeTileBitmask(const std::vector<uint8_t>& grid, int x, int y, int gridWidth) {
    auto IsSolid = [&](int cx, int cy) -> uint8_t {
        if (cx < 0 || cx >= gridWidth || cy < 0) return 1;
        return grid[cy * gridWidth + cx] > 0 ? 1 : 0;
    };

    uint8_t mask = 0;
    if (IsSolid(x, y - 1)) mask |= 1; // North
    if (IsSolid(x + 1, y - 1)) mask |= 2; // North-East
    if (IsSolid(x + 1, y)) mask |= 4; // East
    if (IsSolid(x + 1, y + 1)) mask |= 8; // South-East
    if (IsSolid(x, y + 1)) mask |= 16; // South
    if (IsSolid(x - 1, y + 1)) mask |= 32; // South-West
    if (IsSolid(x - 1, y)) mask |= 64; // West
    if (IsSolid(x - 1, y - 1)) mask |= 128; // North-West
    return mask;
}

```

```
}
```

6.2 GLSL Shaders

1. Bioluminescent Bloom Shader ([bioluminescence.frag](#))

```
#version 330 core
in vec2 FragTexCoord;
out vec4 FragColor;

uniform sampler2D sceneTexture;
uniform float time;
uniform vec3 glowColor; // Cyan, Magenta, or Acid Green

void main() {
    vec4 texColor = texture(sceneTexture, FragTexCoord);

    // Calculate pulse based on time
    float pulse = 0.5 + 0.5 * sin(time * 3.0);

    // High-intensity pixels emit bioluminescent aura
    float brightness = dot(texColor.rgb, vec3(0.299, 0.587, 0.114));
    vec3 glow = vec3(0.0);

    if (brightness > 0.6) {
        glow = glowColor * pulse * (brightness - 0.6) * 2.5;
    }

    FragColor = vec4(texColor.rgb + glow, texColor.a);
}
```

2. Hydrodynamic Water Caustics Shader ([water_caustics.frag](#))

```
#version 330 core
in vec2 FragTexCoord;
out vec4 FragColor;

uniform sampler2D sceneTexture;
uniform float time;

void main() {
    vec2 coord = FragTexCoord;
```

```

// Underwater distortion offset
float distortionX = sin(coord.y * 25.0 + time * 2.5) * 0.002f;
float distortionY = cos(coord.x * 20.0 + time * 2.0) * 0.002f;
coord += vec2(distortionX, distortionY);

// Dynamic Caustic Lattice Pattern
float caustic = sin(coord.x * 50.0 + coord.y * 50.0 + time * 3.5) *
               cos(coord.x * 25.0 - coord.y * 35.0 + time * 1.8);
caustic = clamp(pow(caustic + 0.75, 3.0), 0.0, 1.0) * 0.2;

vec4 sceneColor = texture(sceneTexture, coord);
vec3 waterTint = vec3(0.02, 0.18, 0.28); // Deep Teal Tint

FragColor = vec4(mix(sceneColor.rgb, waterTint, 0.20) + vec3(caustic), sceneColor.a);
}

```

3. Abyssal Pressure Vignette Shader (`pressure_vignette.frag`)

```

#version 330 core
in vec2 FragTexCoord;
out vec4 FragColor;

uniform sampler2D sceneTexture;
uniform float depth; // Depth in meters down the trench

void main() {
    vec4 texColor = texture(sceneTexture, FragTexCoord);

    // Calculate distance from screen center
    vec2 uv = FragTexCoord - vec2(0.5);
    float dist = length(uv);

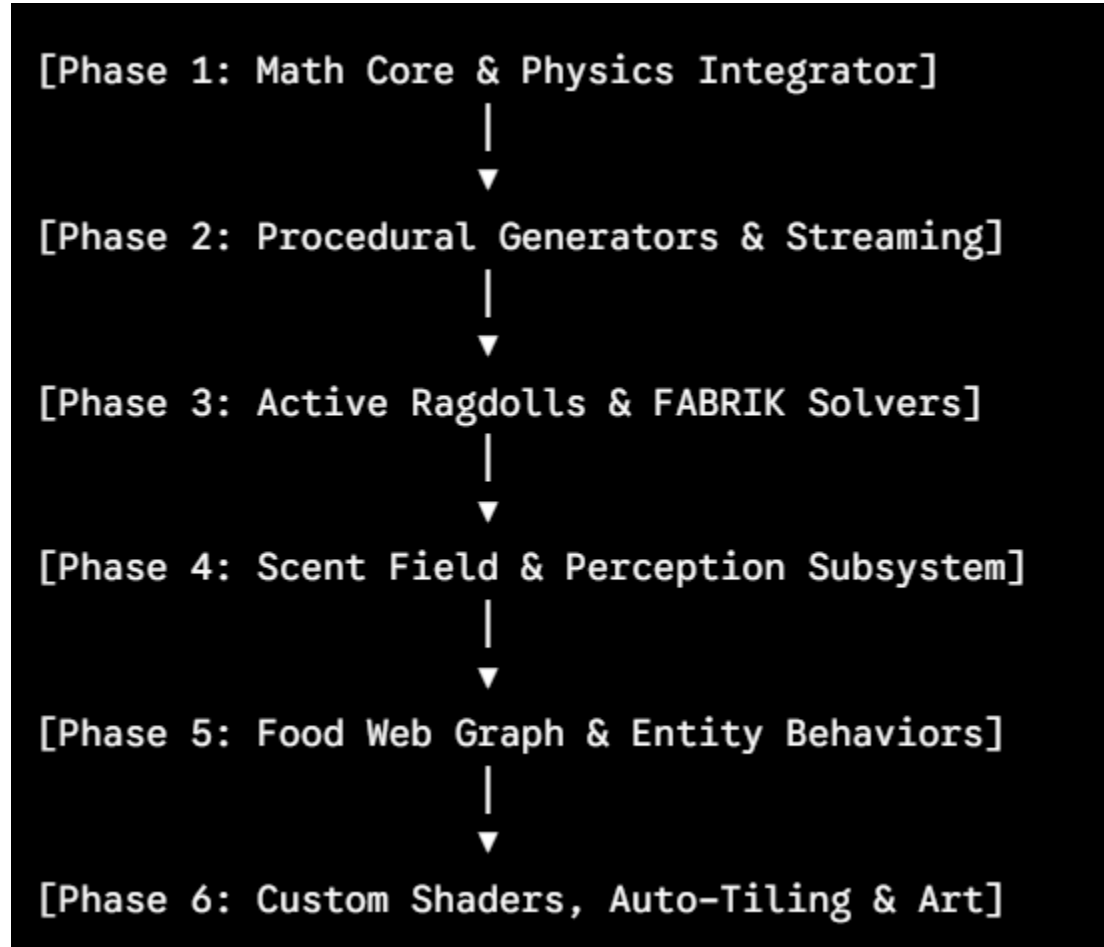
    // Scale vignette intensity with depth
    float intensity = clamp(depth / 2000.0, 0.0, 1.0);
    float vignette = smoothstep(0.7 - (intensity * 0.3), 0.2, dist);

    FragColor = vec4(texColor.rgb * vignette, texColor.a);
}

```

7. GRANULAR IMPLEMENTATION ROADMAP FOR CLAUDE CODE (OPUS 5.5 IN C++)

Claude Code must implement this C++ codebase in **6 distinct technical phases**.



Phase 1: Physics Engine Core & Player Controller

- **Step 1.1:** Create `PhysicsEngine.hpp/cpp`. Implement `PhysicsIntegrator` using Velocity-Verlet integration.
- **Step 1.2:** Implement `PlayerPhysicsController` supporting transitions across traversal states: Sprint, Slide, Impact Roll, Hydro-Glide, Water Dash, and Parachute Brake.
- **Step 1.3:** Implement narrowphase Continuous Collision Detection (CCD) against solid tiles and friction materials.

Phase 2: Procedural Terrain & Chunk Engine

- **Step 2.1:** Implement `CellularAutomata` generator for Cave and Pipe levels.
- **Step 2.2:** Build vertical streaming chunk pipeline for *The Open Abyss* with sliding generation windows.

- **Step 2.3:** Write Poisson Disc Sampling spawner for **CreatureDen** placement.

Phase 3: Active Ragdolls & IK System

- **Step 3.1:** Implement **ConstrainedFABRIKSolver** with parent-relative angle constraints.
- **Step 3.2:** Write procedural locomotion algorithms: Serpentine Sine Wave, Multipod Tripod Gait, Hydrodynamic Swimming Forces, and Biped PD Motors.
- **Step 3.3:** Implement **RagdollRig** dynamic limb severance system triggering severed body physics on high impact forces.

Phase 4: Scent Grid & Perception Engine

- **Step 4.1:** Implement downsampled Eulerian **ScentGrid** with discrete diffusion, decay, and advection.
- **Step 4.2:** Build **PerceptionSystem** computing visual raycast cones, acoustic wave attenuation, and scent gradients.

Phase 5: Interconnected Food Web & AI Behaviors

- **Step 5.1:** Implement **FoodWebGraph** managing directed entity relationships, dynamic preference scores, and corpse events.
- **Step 5.2:** Implement **AIBehaviorEngine** handling the 12 Abnormal Personality overrides.
- **Step 5.3:** Program procedural behavior routines for all 80 distinct entities across the 8 biomes.

Phase 6: Shaders, Auto-Tiling & Final Polish

- **Step 6.1:** Build 8-bit dual-grid bitmask auto-tiling map decorator.
- **Step 6.2:** Integrate GLSL custom shaders (**bioluminescence.frag**, **water_caustics.frag**, **pressure_vignette.frag**).
- **Step 6.3:** Build multi-layer parallax scrolling background pipeline.

With prompt to further develop the AI

To successfully pivot your platforming section from the tight, arcade-style physics of *Super Meat Boy* to a procedural, ecosystem-driven experience, your C++ architecture needs a fundamental overhaul. Instead of static hitboxes and predetermined enemy paths, Claude will need to build a physics-driven animation system and a state-machine-based AI for the creatures.

Here is the structural blueprint to feed Claude for this update.

1. Parkour Level Expansion

Your current platforming roster of The Pipes, The Hull, and The Pirate Ship will be expanded using the procedural generation logic already in place, but re-skinned and mechanically adapted to fit the main rogue-like biomes.

PDF+ 1

- **The Island:** Procedural generation should favor open-air gaps and tribal structures, with hazards based on tribal weapons and hostile dogs.
PDF
- **The Cave:** The algorithm must generate tight, claustrophobic tunnels requiring slides and fast-falls, populated by mutated crustaceans and worms.
PDF
- **The Weeds:** This map will rely heavily on verticality, using dense kelp stalks as climbable poles. Players will need to navigate around mermen, barracudas, and fungal hazards.
PDF+ 1
- **Atlantis:** Generate decrepit, Roman-style city ruins. This level should require precise wall-jumping and ledge-grabbing to avoid the trapped "lost ones".
PDF+ 1
- **The Open Abyss:** A completely new biome focused on an endless vertical descent, requiring the player to utilize free-falling and momentum-saving rolls to survive massive drops.

2. Physics-Based Movement (Rain World Style)

To replicate the weighty, momentum-based movement of *Rain World*, Claude must implement a physics controller that prioritizes momentum conservation and contextual states over rigid velocity caps.

- **Slides and Pounces:** Implement a sliding state triggered by pressing down and jump while moving. If the player jumps mid-slide, the physics engine should apply a massive horizontal velocity multiplier, creating a "slide pounce".
PDF+ 1

- **Impact and Rolling:** Falling from heights greater than a standard jump must apply a landing stun. Claude needs to code a frame-window where holding down and a directional input upon impact converts this downward velocity into a forward roll, negating the stun.
PDF+ 1
- **Pole Mechanics:** Introduce climbable poles where the player can grab, slide down, balance on the tips, and execute momentum-boosting backflips. Pressing jump while holding forward on a horizontal pole should initiate a "pole hop" for faster traversal.
PDF+ 1
- **Aquatic Mobility:** Because your game is centered around a submarine, Claude should implement fluid dynamics where the player drifts upward naturally. Swimming horizontally should require rhythmic inputs (wriggling) for speed, and jumping underwater should provide a speed boost at the cost of an oxygen meter.
PDF+ 2

3. Dynamic Ecosystem AI

Enemies can no longer be static hazards. Claude must program them as living organisms that interact with each other just as much as they interact with the player.

- **The Den System:** Creatures should spawn from designated dens (which can be off-screen) and retreat to them when injured or full. You can also implement a lineage system where killing a creature gives a statistical chance for a harder, mutated variant to spawn from that den on the next run.
PDF+ 1
- **Predator and Prey Logic:** Give every entity in the level a hierarchy tag. Larger predators will actively hunt the player, but the AI state machine must constantly check for closer, easier prey. If a lesser prey item enters the predator's detection radius, it should abandon the player to hunt the easier meal.
PDF+ 1
- **Pack Dynamics:** Create a specific AI class for smaller, weaker creatures. While alone they will flee, but when their local proximity detects a group of their own kind, their state should switch to aggressive, allowing them to coordinate and swarm larger predators.
PDF
- **Procedural Personalities:** When a creature is spawned, Claude should use an ID system to assign randomized statistics. This ID will permanently alter their variables for aggression, intelligence, and passive/hostile behavior so that no two runs feature the exact same animal behavior.
PDF+ 1

How do you want to handle the transition between the turn-based rogue-like combat and this physics-heavy platforming—should the characters share the exact same health pools and stats across both modes, or operate independently?

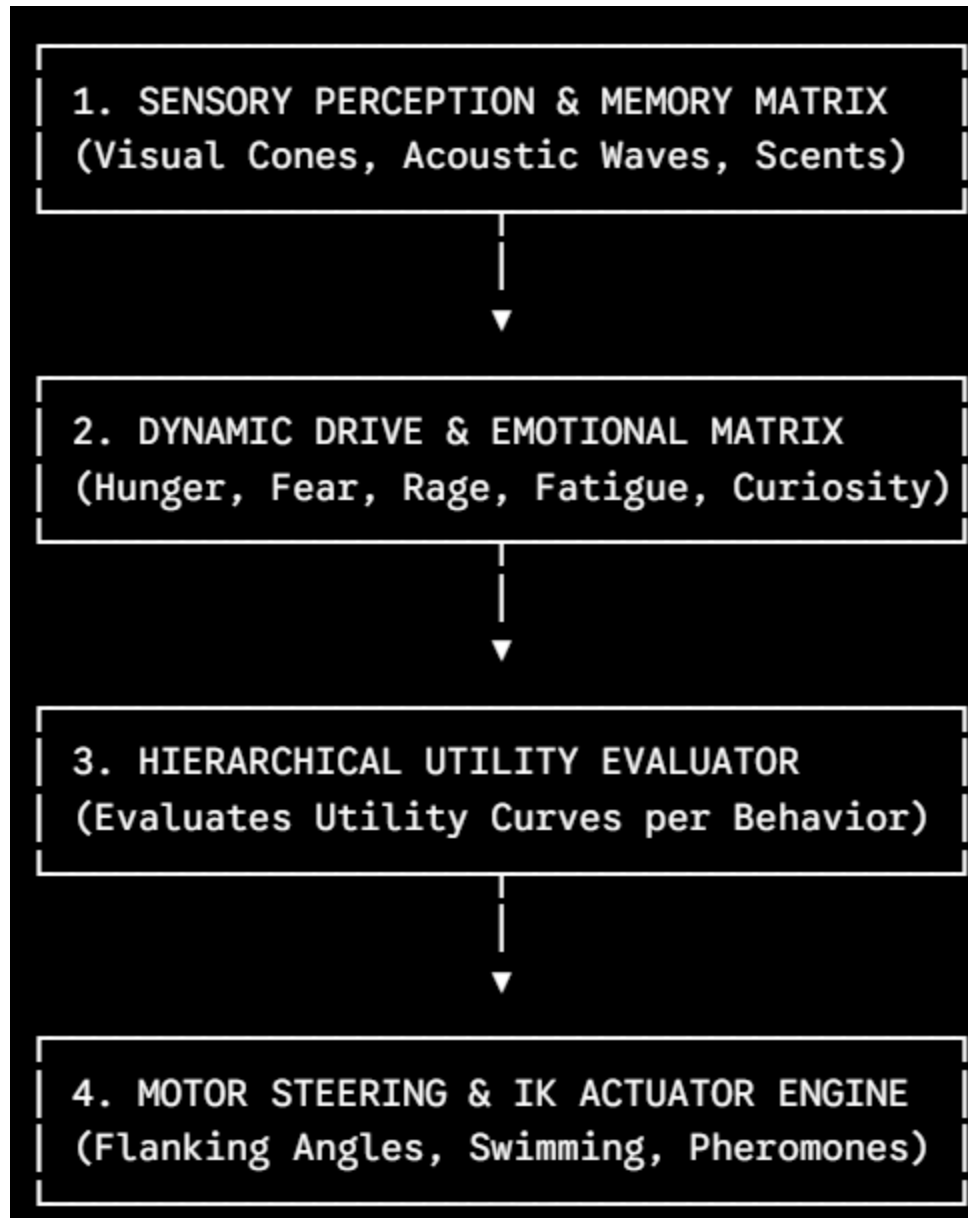
DIRECTIONS1: ADVANCED BEAST AI ARCHITECTURE & ECOSYSTEM INTELLIGENCE ENGINE

Target Implementation: C++17 / C++20 Ecosystem Subsystem for SFML/SDL2
Engine Pipeline

LLM / Claude Code Execution Directives: Implement the structures and algorithms below without omitting behavioral states, simplifying utility curves into rigid boolean switches, or substituting complex sensory models with simple distance checks.

1. MULTI-LAYERED BEAST AI PIPELINE

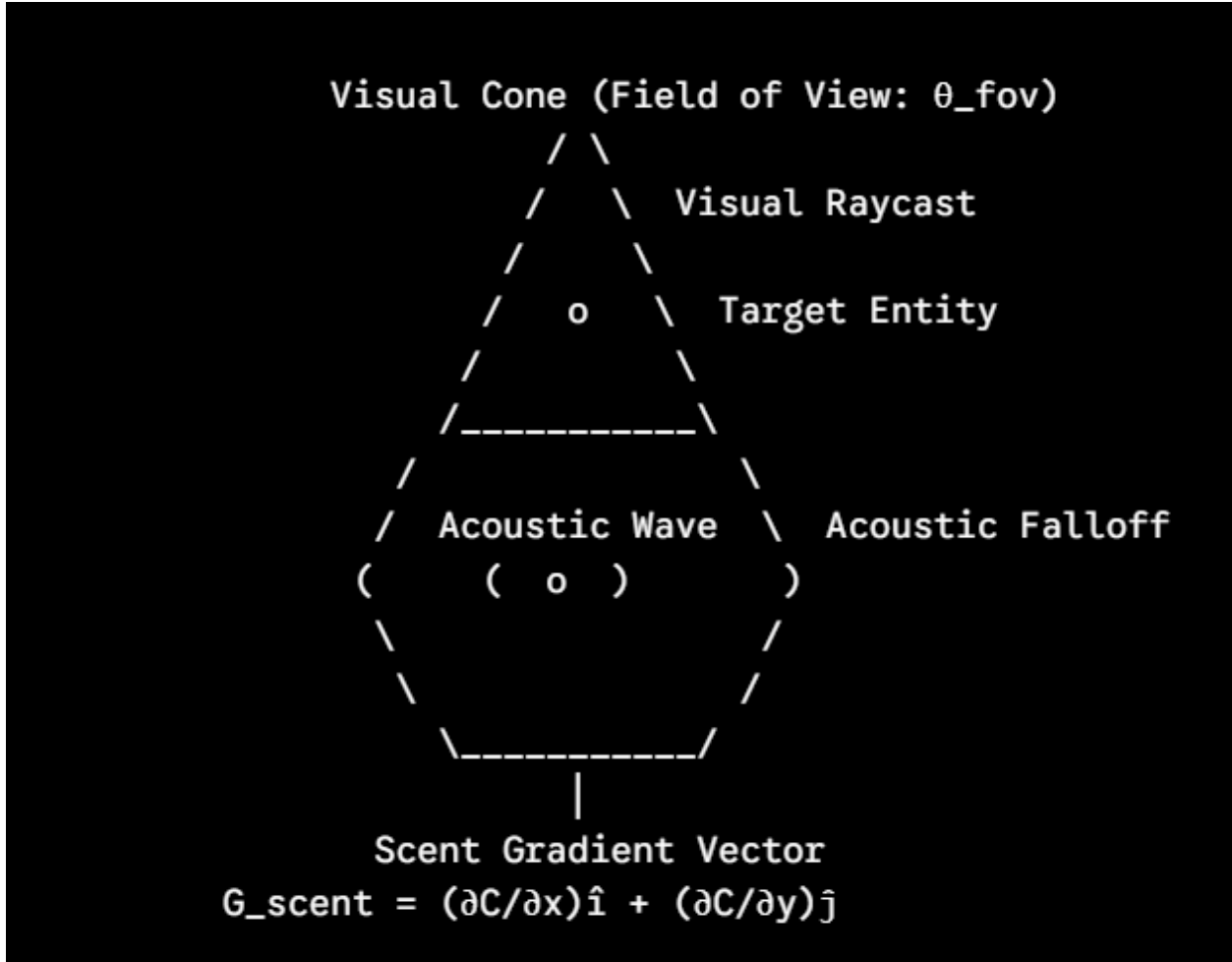
The Beast AI engine does not rely on simple state machines or linear `if-else` checks. Each beast operates via a four-tier cognitive processing pipeline executed every tick:



2. CORE SENSORY, MEMORY & UTILITY MATHEMATICS

2.1 Sensory Perception Matrix

Each beast calculates perception through three distinct spatial fields:



Visual Optics: Evaluated via raycasting inside a dynamic FOV cone (θ_{fov}). Attenuation scales with ambient water clarity α_{water} and target speed v_{target} :

$$S_{\text{visual}} = \text{Big}(1.0 - \frac{d}{R_{\text{max}}}) \cdot \cos(\theta) \cdot \alpha_{\text{water}} \cdot \text{Big}(1.0 + k_{\text{motion}} \cdot \|\vec{v}_{\text{target}}\|)$$

Acoustic Wave Receptor: Detects sound pulses generated by running, water dashes, wall impacts, or explosions. Attenuates quadratically over distance d :

$$S_{acoustic} = \frac{I_{source}}{1.0 + k_{damp} \cdot d^2}$$

Hydro-Scent Vector: Samples the Eulerian Scent Grid gradient $\vec{G}_{scent}$ at the entity's position to determine the directional vector $\hat{u}_{scent}$ toward blood, spores, or rotting

$$\hat{u}_{scent} = \frac{\vec{G}_{scent}}{\|\vec{G}_{scent}\| + \epsilon}$$

2.2 Short-Term Memory Buffer & Traumatic Learning
Beasts retain a spatial memory array MemoryBuffer of up to 16 recent stimuli (threats, food sources, loud noise locations). Memory intensity decays exponentially over time:

$$M(t) = M_0 \cdot e^{-\lambda \cdot (t - t_0)}$$

If a beast receives severe damage ($> 30\%$ max HP) from a specific entity or weapon type (e.g., electrified spears), it writes an entry into its Traumatic Memory Matrix, permanently lowering its bravery threshold toward that entity species and increasing its reaction utility for Flee or Ambush.

2.3 Non-Linear Utility Response Curves

Utility values $U(A) \in [0.0, 1.0]$ for competing behaviors $A \in \{Hunt, Flee, Ambush, Rest, CallPack, Scavenge\}$ are derived using normalized Sigmoidal and Logistic response curves:

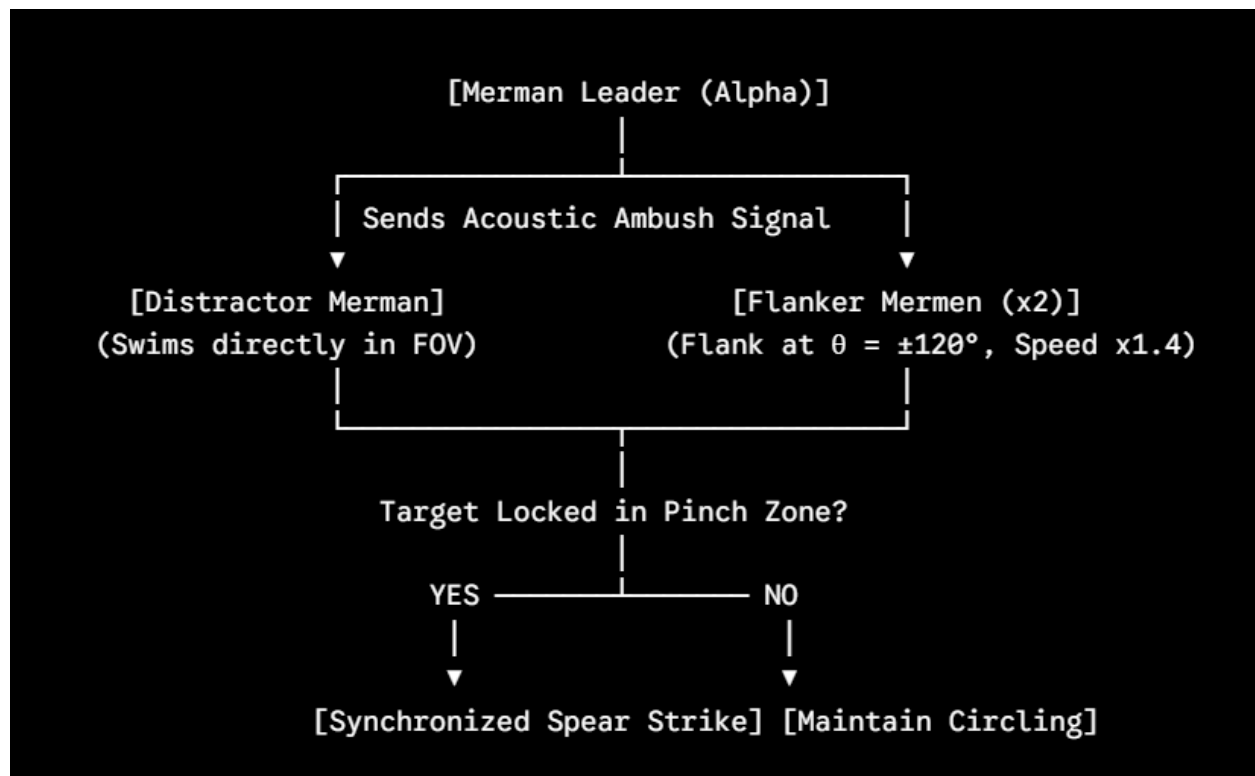
$$U_{Sigmoid}(x, k, m) = \frac{1}{1 + e^{-k(x-m)}}$$

Where x is the input driver (e.g., Hunger level $\in [0, 1]$), k controls curve steepness, and m represents the inflection midpoint.

3. DEEP CREATURE AI PROFILES & ALGORITHMIC SPECIFICATIONS

3.1 The Merman Tide-Hunter (Tactical Pack Ambush & Flanking AI)

Behavioral Profile Mermen are social, highly intelligent aquatic pack hunters. They never engage in head-on solitary rushes unless cornered. They use acoustic click signals to coordinate encirclement tactics, sending a high-health "Distractor" to draw the target's visual cone while "Flankers" position themselves behind the prey.



C++ Tactical Flanking Implementation ([MermanAIController.hpp](#))

```

#pragma once
#include <SFML/System/Vector2.hpp>
#include <vector>
#include <cmath>
#include <algorithm>

```

```
enum class MermanRole { AlphaLeader, Distractor, FlankerLeft, FlankerRight, Retreating };
```

```
struct MermanPack {  
    uint32_t packID;  
    sf::Vector2f targetLastKnownPos;  
    bool targetEncircled = false;  
    float packRage = 0.0f;  
};
```

```
class MermanAIController {  
public:  
    MermanRole currentRole = MermanRole::FlankerLeft;  
    float spearCooldown = 0.0f;  
    float acousticSignalTimer = 0.0f;  
  
    sf::Vector2f ComputeSteeringVelocity(  
        sf::Vector2f selfPos,  
        sf::Vector2f targetPos,  
        sf::Vector2f targetForward,  
        MermanPack& pack,  
        float dt  
    ) {  
        sf::Vector2f toTarget = targetPos - selfPos;  
        float dist = std::sqrt(toTarget.x * toTarget.x + toTarget.y * toTarget.y);  
        sf::Vector2f dirToTarget = (dist > 0.001f) ? toTarget / dist : sf::Vector2f{0.0f, 0.0f};  
  
        // 1. Role Allocation Logic  
        sf::Vector2f desiredPosition;  
        float targetDistanceOffset = 3.0f; // 3 meters separation  
  
        switch (currentRole) {  
            case MermanRole::Distractor: {  
                // Positions directly in target's forward view cone to draw attention  
                desiredPosition = targetPos + targetForward * targetDistanceOffset;  
                break;  
            }  
            case MermanRole::FlankerLeft: {  
                // Rotates target forward vector by +120 degrees  
                sf::Vector2f flankDir{-targetForward.y * 0.866f - targetForward.x * 0.5f,  
                                     targetForward.x * 0.866f - targetForward.y * 0.5f};  
                desiredPosition = targetPos + flankDir * targetDistanceOffset;  
                break;  
            }  
        }  
    }  
};
```

```

    case MermanRole::FlankerRight: {
        // Rotates target forward vector by -120 degrees
        sf::Vector2f flankDir{targetForward.y * 0.866f - targetForward.x * 0.5f,
                             -targetForward.x * 0.866f - targetForward.y * 0.5f};
        desiredPosition = targetPos + flankDir * targetDistanceOffset;
        break;
    }
    case MermanRole::AlphaLeader: {
        // Coordinates team from medium range, moves in to strike when flankers are in
position
        desiredPosition = targetPos - targetForward * (targetDistanceOffset * 1.5f);
        break;
    }
    case MermanRole::Retreating: {
        return -dirToTarget * 400.0f; // Rapid escape impulse
    }
}

// 2. Compute Steering Vector to Desired Tactical Position
sf::Vector2f steeringForce = desiredPosition - selfPos;
float steeringDist = std::sqrt(steeringForce.x * steeringForce.x + steeringForce.y *
steeringForce.y);

if (steeringDist > 0.1f) {
    steeringForce = (steeringForce / steeringDist) * 280.0f; // Swim speed
}

// 3. Attack Trigger Check: If in position and spear ready, launch thrust attack
spearCooldown -= dt;
if (dist < 1.8f && spearCooldown <= 0.0f) {
    spearCooldown = 2.5f; // Reset attack timer
    return dirToTarget * 600.0f; // Attack thrust vector
}

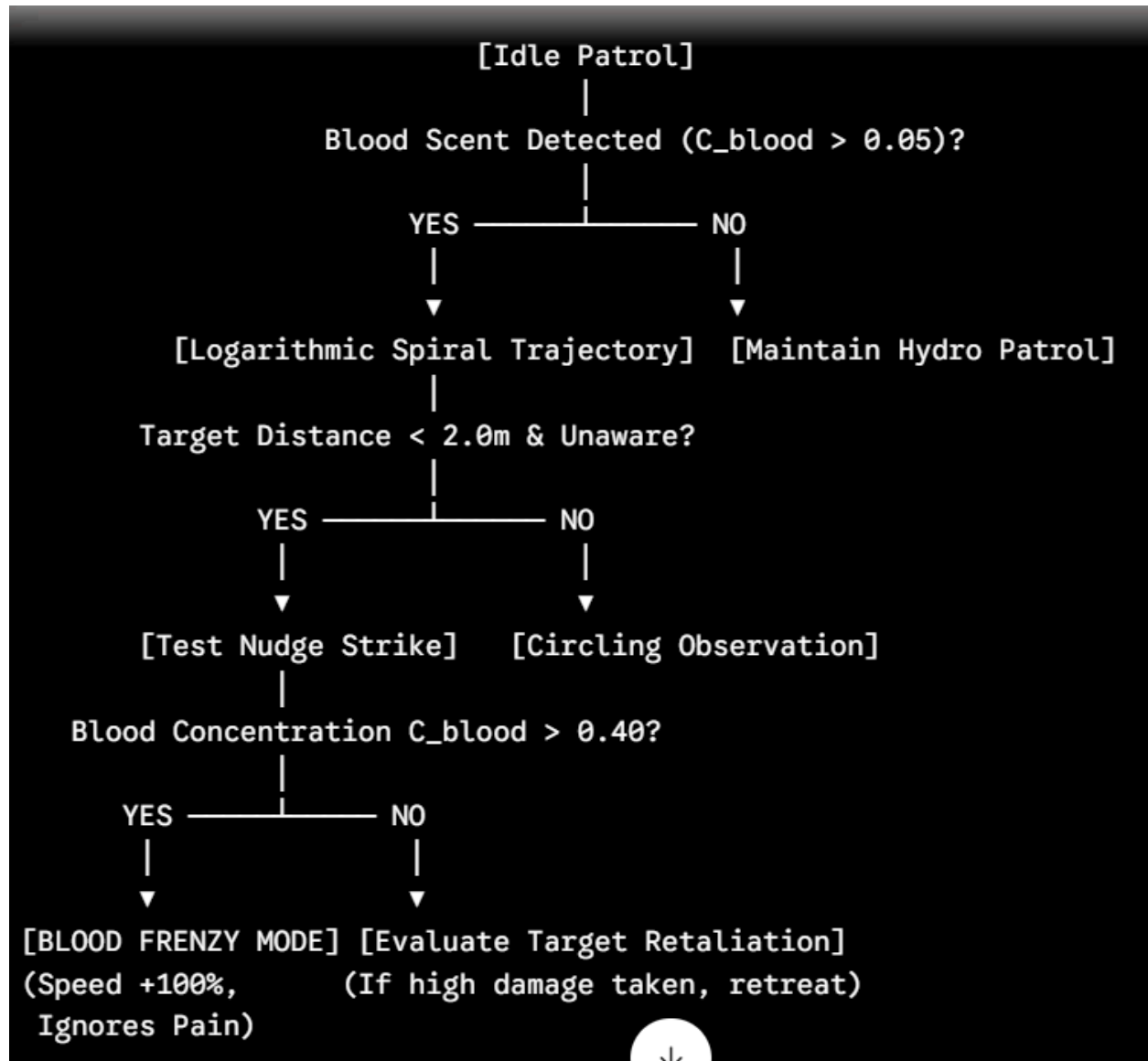
return steeringForce;
}
};

```

3.2 The Tiger Shark (Opportunistic Scent-Tracking & Frenzy Apex AI)

Behavioral Profile

The Tiger Shark is a solo apex predator driven almost entirely by the Eulerian Scent Grid. It uses logarithmic spiral trajectories when investigating scent trails, testing prey defenses with non-lethal "nudges" before entering an unstoppable **Blood Frenzy** state when blood scent levels surpass a critical threshold.



Spiral Approach Trajectory Equation

The approach trajectory uses a logarithmic spiral centered on the target position (x_t, y_t) :

$$r(\theta) = a \cdot e^{b \cdot \theta}$$

Where r is the distance to target, θ is the cumulative orbital angle, a is the initial approach radius, and $b = -0.15$ causes a steady inward spiral.

C++ Blood Frenzy Logic (**TigerSharkAIController.hpp**)

```
#pragma once
#include <SFML/System/Vector2.hpp>
#include <cmath>
#include <algorithm>

class TigerSharkAIController {
public:
    enum class SharkState { Patrol, ScentTracking, TestingNudge, BloodFrenzy, Retreating };
    SharkState state = SharkState::Patrol;

    float spiralAngle = 0.0f;
    float frenzyTimer = 0.0f;
    float health = 250.0f;
    float maxHealth = 250.0f;

    sf::Vector2f UpdateSharkBehavior(
        sf::Vector2f sharkPos,
        sf::Vector2f targetPos,
        float localBloodScent,
        sf::Vector2f scentGradient,
        float dt
    ) {
        // 1. Evaluate State Transitions based on Scent and Damage
        if (health < maxHealth * 0.25f && state != SharkState::BloodFrenzy) {
            state = SharkState::Retreating; // Low health escape unless in frenzy
        } else if (localBloodScent > 0.40f || (state == SharkState::TestingNudge && localBloodScent
        > 0.15f)) {
            state = SharkState::BloodFrenzy;
            frenzyTimer = 8.0f; // 8 seconds of relentless attack
        } else if (localBloodScent > 0.05f && state == SharkState::Patrol) {
            state = SharkState::ScentTracking;
        }

        // 2. State Actions
        sf::Vector2f toTarget = targetPos - sharkPos;
        float dist = std::sqrt(toTarget.x * toTarget.x + toTarget.y * toTarget.y);

        switch (state) {
            case SharkState::Patrol: {
```

```

// Wide, smooth sinusoidal swimming trajectory
spiralAngle += dt * 0.5f;
return sf::Vector2f{std::cos(spiralAngle), std::sin(spiralAngle)} * 80.0f;
}

case SharkState::ScentTracking: {
    // Follows scent gradient vector, transition to logarithmic spiral near target
    if (dist > 6.0f) {
        float gradMag = std::sqrt(scentGradient.x * scentGradient.x + scentGradient.y *
scentGradient.y);
        return (gradMag > 0.001f) ? (scentGradient / gradMag) * 180.0f : toTarget / dist *
180.0f;
    } else {
        // Logarithmic Spiral trajectory computation
        spiralAngle += dt * 1.8f;
        float radius = 6.0f * std::exp(-0.15f * spiralAngle);
        sf::Vector2f spiralOffset{std::cos(spiralAngle) * radius, std::sin(spiralAngle) * radius};
        sf::Vector2f desiredPos = targetPos + spiralOffset;

        if (dist < 1.5f) state = SharkState::TestingNudge;
        return (desiredPos - sharkPos) * 2.5f;
    }
}

case SharkState::TestingNudge: {
    // Fast direct bump to gauge prey response/mass
    return (toTarget / dist) * 350.0f;
}

case SharkState::BloodFrenzy: {
    // Maximum speed, direct trajectory, ignores turn damping
    frenzyTimer -= dt;
    if (frenzyTimer <= 0.0f) state = SharkState::ScentTracking;
    return (toTarget / dist) * 550.0f; // Extreme frenzy speed
}

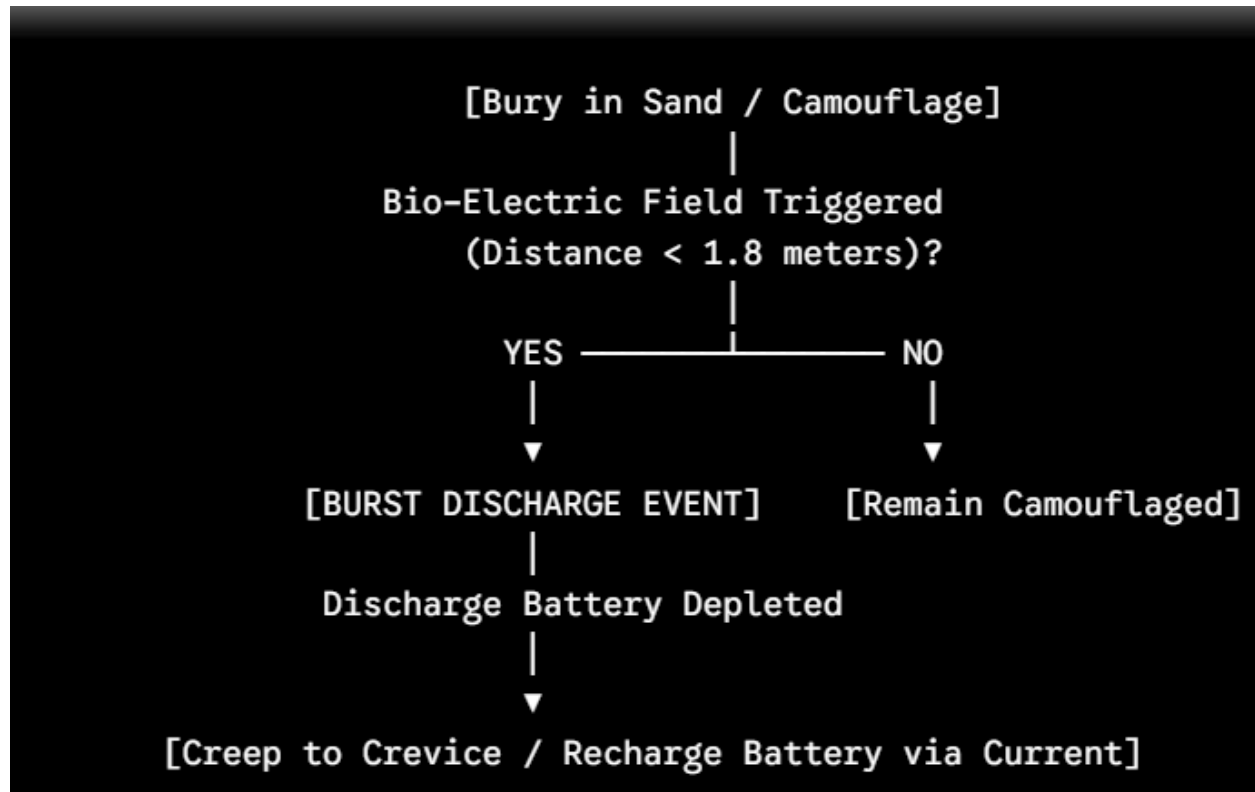
case SharkState::Retreating: {
    return -(toTarget / dist) * 300.0f;
}
}
return {0.0f, 0.0f};
};

```

3.3 The Electric Ray (Camouflage Lurker & Kinetic Hazard Trap AI)

Behavioral Profile

The Electric Ray is a passive-aggressive ambush predator that buries itself in sea-floor silt or clings to dark trench walls. It tracks targets using bio-electric field receptors (passive sensing through walls). When a target passes within its discharge radius $R_{discharge}$, it releases a high-voltage shock wave that stuns prey, damages metals, and ignites nearby powder/gas pockets.



C++ Kinetic Discharge & Trap Controller ([ElectricRayAIController.hpp](#))

```
#pragma once
#include <SFML/System/Vector2.hpp>
#include <cmath>
```

```
struct ShockwaveEvent {
    sf::Vector2f origin;
    float radius = 2.5f; // Meters
    float damage = 45.0f;
    float stunDuration = 2.0f; // Seconds
    bool ignitesGas = true;
```

```

};

class ElectricRayAIController {
public:
    float batteryCharge = 100.0f; // Max 100%
    bool isCamouflaged = true;
    float stealthAlpha = 0.1f; // Sprite opacity (90% invisible)

    bool EvaluateDischargeTrigger(
        sf::Vector2f rayPos,
        sf::Vector2f targetPos,
        float targetMovementSpeed,
        ShockwaveEvent& outEvent
    ) {
        sf::Vector2f diff = targetPos - rayPos;
        float dist = std::sqrt(diff.x * diff.x + diff.y * diff.y);

        // Bio-electric sensing detects targets through line-of-sight obstructions if moving fast
        float effectiveDetectionRadius = 1.8f + (targetMovementSpeed / 200.0f);

        if (dist <= effectiveDetectionRadius && batteryCharge >= 80.0f) {
            // Trigger Electric Shockwave Discharge
            batteryCharge = 0.0f; // Deplete battery
            isCamouflaged = false; // Unveil
            stealthAlpha = 1.0f;

            outEvent.origin = rayPos;
            outEvent.radius = 2.8f;
            outEvent.damage = 50.0f;
            outEvent.stunDuration = 2.5f;
            return true; // Discharge triggered
        }
        return false;
    }

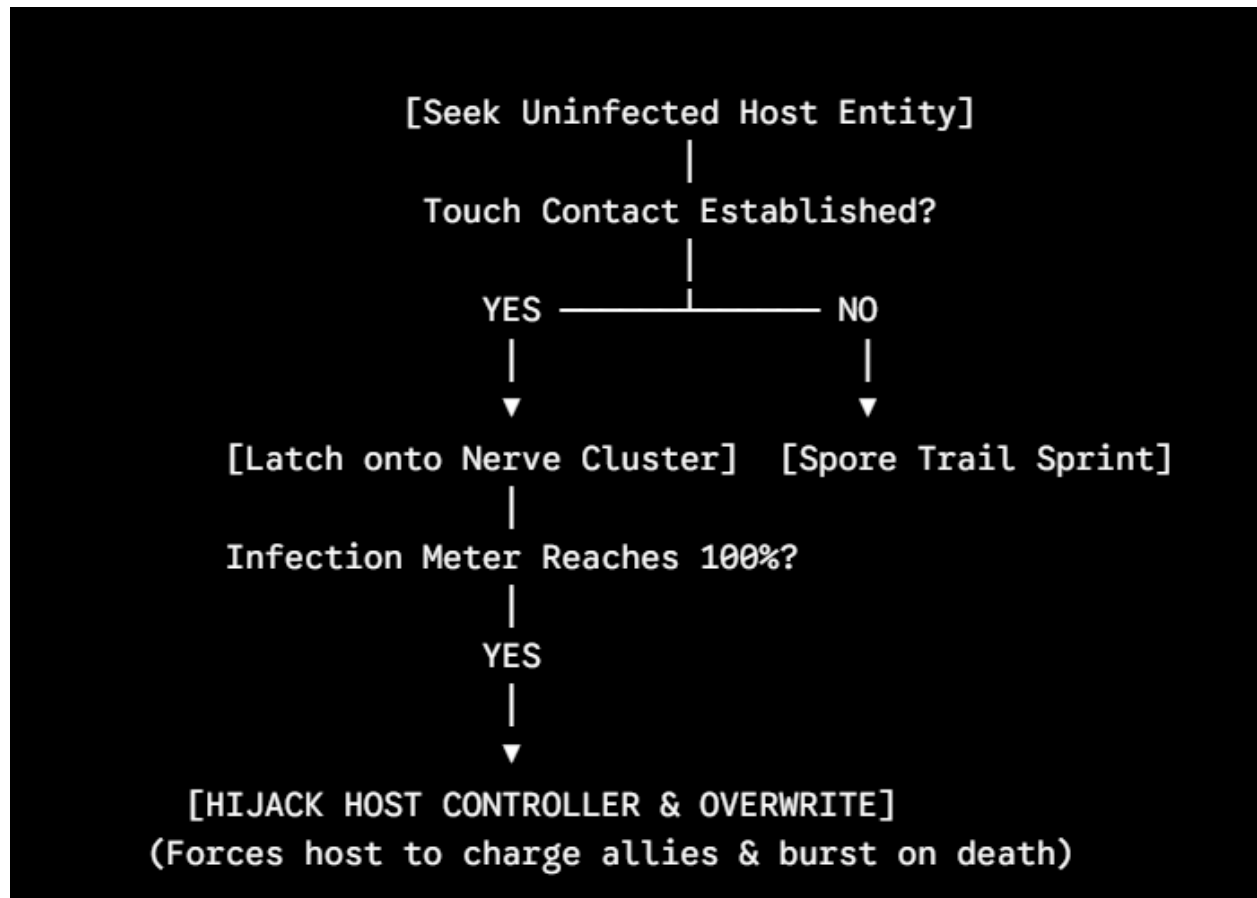
    void Recharge(float fluidCurrentVelocity, float dt) {
        if (batteryCharge < 100.0f) {
            // Kinetic energy from fluid currents recharges the internal battery
            batteryCharge += (10.0f + fluidCurrentVelocity * 0.05f) * dt;
            batteryCharge = std::min(100.0f, batteryCharge);
        }
    }
};

```

3.4 The Parasitic Fungal Beetle (Mind-Control Host Hijacker AI)

Behavioral Profile

The Parasitic Fungal Beetle carries active spore sacs on its carapace. It targets medium-sized fauna or the player. Upon contact, it latches onto the target's nerve cluster, taking control of the host's movement vectors and forcing the host to attack former allies or walk directly into spore clouds to spread the infection.



C++ Host Hijack Machine ([FungalParasiteAIController.hpp](#))

```
#pragma once
#include <SFML/System/Vector2.hpp>
#include <cmath>

struct TargetHost {
    sf::Vector2f position;
    sf::Vector2f velocity;
    float infectionLevel = 0.0f; // 0.0 to 1.0
    bool isFullyControlled = false;
```

```

    uint32_t hostID;
};

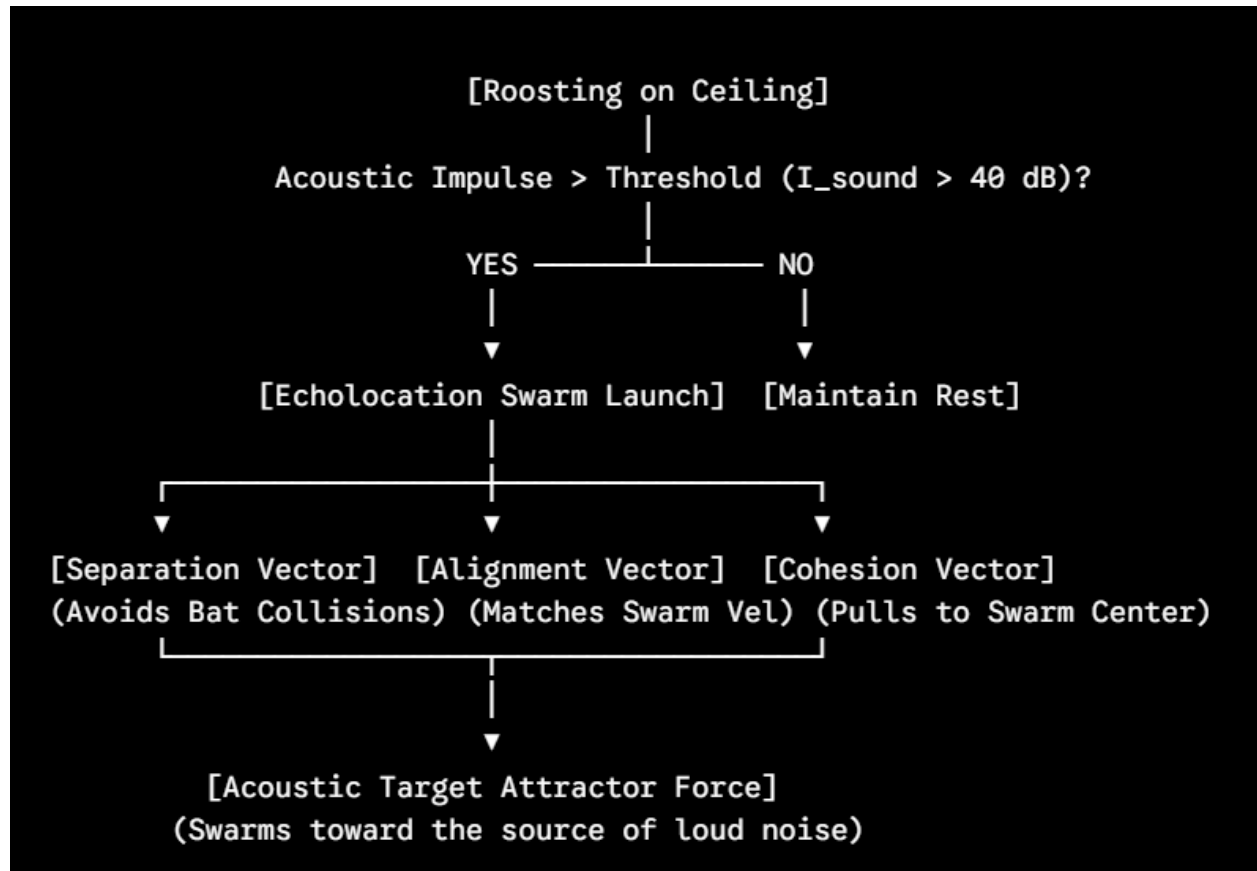
class FungalParasiteAIController {
public:
    bool isLatched = false;
    uint32_t latchedHostID = 0;

    void UpdateInfection(TargetHost& host, float dt, sf::Vector2f& outHostForcedVelocity) {
        if (!isLatched) return;

        if (!host.isFullyControlled) {
            // Inject spores to build up control
            host.infectionLevel += 0.25f * dt; // 4 seconds to full control
            if (host.infectionLevel >= 1.0f) {
                host.isFullyControlled = true;
            }
        } else {
            // FULL HOST OVERWRITE: Drive host toward the nearest uninfected ally group
            outHostForcedVelocity = sf::Vector2f{1.0f, 0.0f} * 220.0f; // Overwrite speed vector
        }
    }
};

```

3.5 The Cave Bat Swarm (Acoustic Boids & Echolocation Navigation AI) Behavioral Profile Cave Bats operate as a single distributed intelligence using modified Craig Reynolds Boids Algorithms augmented with acoustic echolocation raycasts. They linger in roosts near cavern ceilings until sound waves surpass $I_{acoustic} > 40dB$, triggering a panicked swarm response that disorients targets, blocks light sources, and knocks players off ledges.



C++ Acoustic Boids Engine ([CaveBatSwarmAIController.hpp](#))

```
#pragma once
#include <SFML/System/Vector2.hpp>
#include <vector>
#include <cmath>
```

```
struct BatBoid {
    sf::Vector2f position;
    sf::Vector2f velocity;
    float echolocationPingTimer = 0.0f;
};
```

```
class CaveBatSwarmAIController {
public:
    std::vector<BatBoid> bats;
    float separationRadius = 0.6f;
    float neighborRadius = 3.0f;

    void UpdateSwarm(sf::Vector2f soundOrigin, float soundIntensity, float dt) {
```

```

for (auto& bat : bats) {
    sf::Vector2f separation{0.0f, 0.0f};
    sf::Vector2f alignment{0.0f, 0.0f};
    sf::Vector2f cohesion{0.0f, 0.0f};
    int neighborCount = 0;

    for (const auto& other : bats) {
        if (&bat == &other) continue;
        sf::Vector2f diff = bat.position - other.position;
        float dist = std::sqrt(diff.x * diff.x + diff.y * diff.y);

        if (dist < separationRadius && dist > 0.001f) {
            separation += (diff / dist) / dist; // Quadratic repulsion
        }
        if (dist < neighborRadius) {
            alignment += other.velocity;
            cohesion += other.position;
            neighborCount++;
        }
    }

    if (neighborCount > 0) {
        alignment /= static_cast<float>(neighborCount);
        cohesion = ((cohesion / static_cast<float>(neighborCount)) - bat.position) * 0.5f;
    }

    // Sound Source Attractor Force
    sf::Vector2f toSound = soundOrigin - bat.position;
    float soundDist = std::sqrt(toSound.x * toSound.x + toSound.y * toSound.y);
    sf::Vector2f soundAttractor = (soundDist > 0.001f) ? (toSound / soundDist) *
(soundIntensity * 2.0f) : sf::Vector2f{0.0f, 0.0f};

    // Calculate Net Acceleration Vector
    sf::Vector2f accel = separation * 2.5f + alignment * 1.0f + cohesion * 1.0f +
soundAttractor * 1.8f;
    bat.velocity += accel * dt;

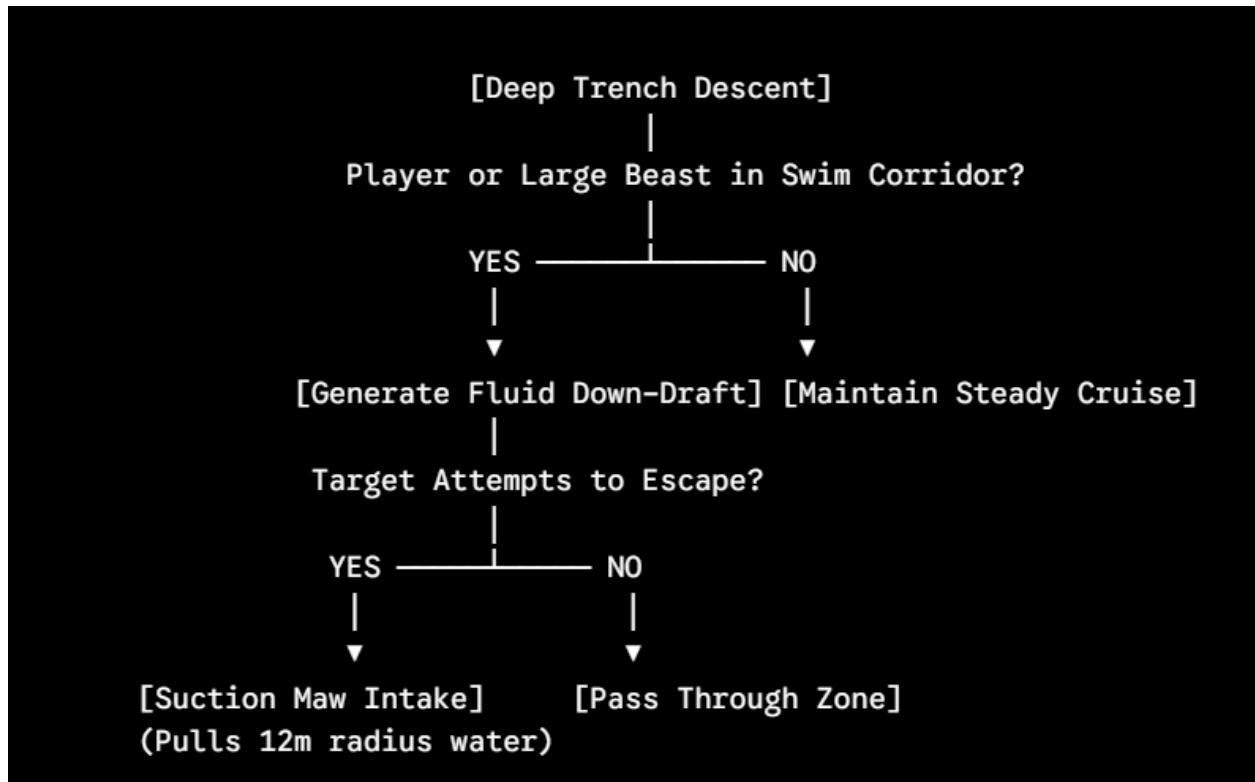
    // Clamp Max Speed
    float speed = std::sqrt(bat.velocity.x * bat.velocity.x + bat.velocity.y * bat.velocity.y);
    if (speed > 350.0f) bat.velocity = (bat.velocity / speed) * 350.0f;

    bat.position += bat.velocity * dt;
}
}

```

};

3.6 The Abyssal Leviathan (Apex Macro-AI & Fluid Draft Controller) Behavioral Profile
The Abyssal Leviathan spans over 20 meters in length and operates as a structural biome hazard. It ignores small prey ($Mass < 100kg$) unless starving. As it swims vertically through The Open Abyss, its body generates high-velocity fluid draft channels that pull players, debris, and smaller beasts along its path.



C++ Macro-Aura & Fluid Force Vector Generator ([LeviathanAIController.hpp](#))

C++

```
#pragma once
#include <SFML/System/Vector2.hpp>
#include <vector>
#include <cmath>
```

```
struct FluidCurrentForce {
    sf::Vector2f position;
    sf::Vector2f forceVector;
    float radius;
```

```

};

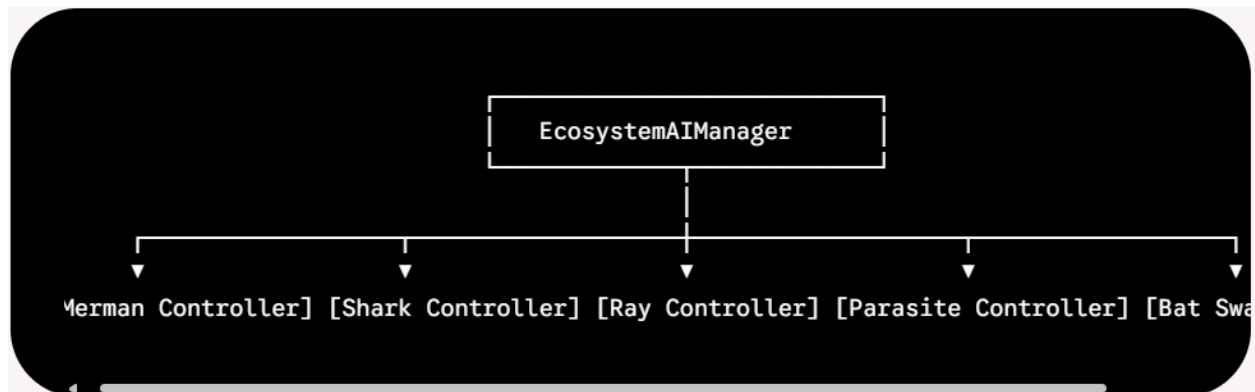
class LeviathanAIController {
public:
    sf::Vector2f headPosition;
    sf::Vector2f velocity{0.0f, 300.0f}; // Fast vertical descent velocity
    float bodyLength = 25.0f;           // 25 meters length

    void GenerateHydrodynamicDraft(std::vector<FluidCurrentForce>& globalFluidForces) {
        // Generates directional draft currents along the entire length of its spine
        for (float offset = 0.0f; offset < bodyLength; offset += 2.0f) {
            FluidCurrentForce draft;
            draft.position = headPosition - sf::Vector2f{0.0f, offset};
            draft.forceVector = velocity * 1.2f; // Downward drag force
            draft.radius = 4.0f;                // 4-meter wide current channel
            globalFluidForces.push_back(draft);
        }
    }
};

```

4. COMPLETE SYSTEM INTEGRATION ARCHITECTURE

All beast controllers are managed by the unified **EcosystemAIManager** class, which connects perception inputs, Eulerian scent grids, pack communication channels, and utility evaluation passes.



4.1 C++ Ecosystem AI Subsystem Coordinator (**EcosystemAIManager.hpp**)

```

#pragma once
#include <vector>
#include <memory>

```

```

#include "MermanAIController.hpp"
#include "TigerSharkAIController.hpp"
#include "ElectricRayAIController.hpp"
#include "FungalParasiteAIController.hpp"
#include "CaveBatSwarmAIController.hpp"
#include "LeviathanAIController.hpp"

```

```

class EcosystemAIManager {

```

```

private:

```

```

    std::vector<MermanAIController> mermen;
    std::vector<TigerSharkAIController> sharks;
    std::vector<ElectricRayAIController> rays;
    FungalParasiteAIController parasiteSystem;
    CaveBatSwarmAIController batSwarm;
    LeviathanAIController leviathan;

```

```

public:

```

```

    void TickEcosystemAI(
        sf::Vector2f playerPos,
        sf::Vector2f playerForward,
        float playerSpeed,
        float ambientLight,
        float dt

```

```

    ) {

```

```

        // 1. Update Merman Pack Tactics

```

```

        MermanPack sharedPack{1, playerPos, false, 0.0f};

```

```

        for (auto& merman : mermen) {

```

```

            sf::Vector2f vel = merman.ComputeSteeringVelocity(

```

```

                merman.spearCooldown > 0.0f ? sf::Vector2f{0.0f, 0.0f} : playerPos - sf::Vector2f{2.0f,
0.0f},

```

```

                playerPos,
                playerForward,
                sharedPack,
                dt

```

```

            );

```

```

        }

```

```

        // 2. Update Tiger Shark Scent Tracking & Frenzy

```

```

        for (auto& shark : sharks) {

```

```

            sf::Vector2f steer = shark.UpdateSharkBehavior(

```

```

                sf::Vector2f{0.0f, 0.0f},

```

```

                playerPos,

```

```

                0.20f, // Sampled blood level

```

```

                sf::Vector2f{0.1f, 0.5f}, // Scent gradient

```

```

        dt
    );
}

// 3. Evaluate Electric Ray Discharge Traps
ShockwaveEvent shock;
for (auto& ray : rays) {
    if (ray.EvaluateDischargeTrigger(sf::Vector2f{10.0f, 10.0f}, playerPos, playerSpeed,
shock)) {
        // Trigger environmental shockwave damage & ignitions
    }
    ray.Recharge(150.0f, dt);
}

// 4. Update Cave Bat Acoustic Swarm
batSwarm.UpdateSwarm(playerPos, playerSpeed > 200.0f ? 60.0f : 10.0f, dt);
}
};

```